Comparison of Kubernetes-based Autoscaling on Proxmox VE

MASTER THESIS

Hof University of Applied Sciences
Graduate School
Software Engineering for Industrial Applications

Submitted to
Prof. Dr. Jürgen Heym
Alfons-Goppel-Platz 1
95028 Hof
Germany

Submitted by
Nishant Singh Chauhan
Fabrikzeile 26/01
95028 Hof
Germany

Hof, May 01, 2026

Abstract

Comparison of Kubernetes-based Autoscaling on Proxmox VE

Kubernetes has become the standard choice for container orchestration, however, the autoscaling capabilities of Kubernetes are deeply rooted in public cloud infrastructure, where the API calls made by the cloud vendor themselves take care of node creation and management seamlessly. However, these assumptions do not hold good for a self-managed private cloud infrastructure, wherein there is no standard way of communicating between the Kubernetes Autoscaler and the virtualization platform being used. The aim of this thesis is to fill this gap.

The experiment is conducted as a 2×2 comparative experiment involving two Kubernetes versions, namely Kubeadm and K3s, each installed on two types of compute infrastructure, Ubuntu VMs and Proxmox LXC. Four different cluster configurations result from this. The same procedure of infrastructure provisioning is used across all four configurations using OpenTofu and its provider bpg/Proxmox, thereby excluding any possible differences in results because of provisioning process. The proposed external gateway layer design consists of FreeBSD/PF, HAProxy, and FRRouting with MetalLB, and the proposed architecture involves using BGP route synchronization, which is achieved with MetalLB Layer 2 (ARP) failover as described in Chapter 5. Monitoring and observability of the system are provided by Prometheus and Grafana, and JMeter is used for load testing.

Though both pod-level (Horizontal Pod Autoscaler) and cluster-level (Cluster Autoscaler) solutions have been considered for analysis, this research pays special attention to cluster-level solutions. Current scientific literature provides comprehensive information about the effectiveness of pod-level Autoscaler. Cluster-Autoscaler-related scientific works have mostly focused on node pools in public cloud platforms while the issue of node scaling behavior of cluster Autoscaler on self-managed, physical bare metal hosted private clouds is much less discussed. The thesis presents current practices of the node spawning procedure within four different combinations and analyzes their performance. In this way, the paper provides a SWOT-informed comparative analysis of Kubernetes distributions, virtualization substrate, and infrastructure-as-code approach.

Acknowledgements

Firstly, I would like to give thanks to God Almighty for giving me the strength, guidance, and determination to complete this master's thesis.

Secondly, I would like to give thanks to my supervisor, Prof. Dr. Jürgen HEYM, for his valuable advice, suggestions, and continuous support towards the completion of this thesis and its associated project. His dedication and guidance were crucial towards the completion of this thesis.

Thirdly, I would like to give sincere thanks to my parents, Mrs. Mamta Devi and Shyam Pratap Singh, for their continuous support and encouragement in completing this academic task. I would like to thank them for their sacrifices and patience.

Lastly, I would like to sincerely appreciate everyone who encouraged and supported me in the process of completing this master's thesis.

Table of Contents

Table of Contents....4
List of Abbreviation....8
List of Tables....9
List of Figures....10
1 Introduction....11
1.1 Aim....12
1.2 Objective....12
1.3 Proposed Solution....12
1.3.1 Layer 1: System Architecture Layer....13
1.3.2 Declarative Infrastructure as Code....14
1.3.3 Node Provisioning Tooling....15
1.3.4 Autoscaling Control Loop:....17
1.3.5 SWOT Analyses on Kubernetes Distributions....19
1.3.6 SWOT Analysis of Virtual Machine Based and Container Based Infrastructure....24
1.3.7 SWOT Analysis of the Infrastructure as Code....29
1.3.8 Metrics Collection & Monitoring Infrastructure....32
1.4 Advantages of Proposed Solution....33
2 State of the Art....34
2.1 Introduction....34
2.2 Background and Foundational Technologies....34
2.2.1 Container orchestration and Kubernetes....35
2.2.2 Kubernetes distribution....35
2.2.3 Compute Substrate....36
2.2.4 Autoscaling Mechanism....36

2.2.5 Deeper Look at Kubernetes Architecture 37
2.2.6 The Mathematics of Autoscaling 38
2.2.7 Operating System Internals: Virtual Machines vs. Containers 38
2.3 Existing Research 40
2.3.1 Comparison of Kubernetes Distribution 40
2.3.2 Comparison of Virtualization Substrates 40
2.3.3 Evaluation of Kubernetes Autoscaling 41
2.4 Research Gap and Justification 41
3 System Specification and Requirements 42
3.1 Project Overview 42
3.1.1 Problem Statement Validation 43
3.1.2 Scope and Boundaries 44
3.2 Functional Requirements 45
3.2.1 Infrastructure as Code (IaC) Automation 45
3.2.2 Autoscaling Control Loop 45
3.2.3 Zero-Downtime Network Synchronization 45
3.2.4 Traffic Load Balancer 46
3.3 Non-Functional Requirement 47
3.3.1 Performance 47
3.3.2 Reliability and High Availability 48
3.3.3 Security and Isolation 48
3.4 Monitoring and evaluation Specifications 49
3.4.1 Data Collection Stack 49
3.4.2 Metrics Captured 49
3.4.3 Evaluation Method 50
4 System Design 51
4.1 Architectural Blueprint and Network Topology 51

4.1.1 The Two-Layer Substrate Model 51
4.1.2 Network Topology Flow 52
4.2 Edge Routing and Security Design 53
4.2.1 Perimeter Defense 53
4.2.2 Dynamic Route Synchronization Strategy 53
4.2.3 Load Balancing Architecture 54
4.3 Autoscaling Control Loop 55
4.3.1 Elasticity Logic and Threshold 55
4.3.2 Provisioning Sequence 55
4.4 Experimental design: 2 x 2 Matrix 56
4.4.1 Definitions of the Test Configurations 56
4.4.2 Hardware and environmental Constraint 57
4.4.3 Result of Empirical Study 2 * 2 Matrix 58
4.5 Mechanics of Implementation 62
4.5.1 Implementation of Infrastructure as Code 62
4.5.2 Custom Watcher Middleware 64
4.5.3 MetalLB Layer 2 ARP Operations 65
5 Implementation Complexity & Findings 65
5.1 Substrate and Distribution Incompatibility: The Config-B Finding 65
5.2 Architectural Deviations and the Network Pivot 68
5.3 Time-to-Scale versus Time-to-Recovery: The Latency Recovery Lag 69
5.4 Measurement Validity: The Observer Effect 70
5.5 Selected Engineering Edge-Cases 70
5.6 Storage Exhaustion and Observability Churn 71
5.7 Micro-Phase Decomposition of Time-to-Scale 72
5.8 Control Plane Overhead Analysis 73
5.9 Workload profile specification for JMeter 74

6 Conclusion & Future Work 75
6.1 Discussion 75
6.1.1 The Trade-off Between Security and Performance 75
6.1.2 Total Cost of Ownership (TCO) Implications 76
6.2 Summary of Findings 76
6.3 Limitations 77
6.4 Future Work 77
7 Reference 78

List of Abbreviation

VE	Virtual Environment
VM	Virtual Machine
LXC	Linux Container
LTS	Long term Support
DHCP	Dynamic Host Configuration Protocol
K8s	Kubernetes
K3s	Kubernetes Lightweight Distribution
HPA	Horizontal Pod Autoscaler
VPA	Vertical Pod Autoscaler
CNI	Container Network Interface
etcd	Distributed Key-Value Store
CAPI	Cluster APi
CAPMOX	Cluster API provider Proxmox
BGP	Border Gateway Protocol
eBGP	External Border Gateway Protocol
PF	Packet Filter
TCP	Transmission Control Protocol
FRR	Free Range Routing
IaC	Infrastructure as Code
KPI	Key Performance Metrics
SWOT	Strengths, Weaknesses, Opportunities, Threats
TTS	Time-To-Scale
TTR	Time-To-Recovery
CEI	Core Exhaustion Index
CRI	Container Runtime Interface
ARP	Address Resolution Protocol

List of Tables

Table 1: The 2×2 experimental comparison matrix 14
Table 2: Comparison of node provisioning tools evaluated for the study. 16
Table 3: Scope and Boundaries of the study 44
Table 4: Functional Requirement and their verification criteria 47
Table 5: Non-Functional requirements and how each is specified 48
Table 6: Metrics Captured and their Operational Significance 49
Table 7: Edge gateway traffic. 54
Table 8: The 2 x 2 configuration matrix 57
Table 9: Hardware and environmental baseline 58
Table 10: 2×2 matrix measured Time-to-Scale results. 59
Table 11: Estimate of microphase decomposition of the aggregate Time-to-Scale measured. 72
Table 12: Resource consumption of idle-baseline resources in both control plane architectures 73
Table 13: Apache JMeter load test profile applied to all configurations. 74

List of Figures

Figure 1: End-to-end system architecture across all layers 13
Figure 2: The pod- and cluster-level autoscaling control loop 18
Figure 3: Taxonomy of the design space for Kubernetes autoscaling on a private Proxmox cloud 35
Figure 4: Comparison between Architectural Design of QEMU (Hardware Virtualization) & LXC (Operating System Level Virtualization) 39
Figure 5: Derivation of the System requirements 42
Figure 6: Zero-downtime network synchronization 46
Figure 7: Complete Network Topology Architecture 52
Figure 8: Autoscaling control-loop sequence 56
Figure 9: Grafana Dashboard: Node Scaling, Pending Pods, CPU Usage, and Network Input/Output during Scale Up. 60
Figure 10: Experimental Time to Scale (TTS) Comparison for Various Matrix Configurations. 61
Figure 11: LXC Worker Kubelet Registration Logs (config-b-w0) Successfully Registered with Hosts 67
Figure 12: Nested Flannel CNI Container Log Indicating Socket Permission Error in Unprivileged LXC Environment 67
Figure 13: Aggregate Report of Apache JMeter for Config-D Showing No Connection Failures 69
Figure 14: Grafana Per Node CPU Utilization Showing Latency Recovery Lag Between TTS and TTR. 70

1 Introduction

Kubernetes has proven to be the dominant framework for managing containerized application workloads where the ability to automatically scale up and down based on demand is one of the key features. Managed solutions offered by public clouds like Amazon EKS, Google GKE, and Microsoft AKS provide the solution to this problem. Cloud vendors provide native node pools, and the Kubernetes Cluster Autoscaler has a vendor-specific plugin that converts the decision to scale into a call to the existing provider's API. The part of the process which converts the decision to add another node to the working machine is taken care of by the cloud vendor.

In contrast, in a self-managed private cloud setting, such convenience is not available. In the case where a company uses Kubernetes in its bare-metal hypervisor due to issues of cost, data sovereignty, or control, there is no corresponding provider plugin that can be utilized. The connection between the Kubernetes scaling signal and the virtualization system must be created from scratch. One example of such a scenario is Proxmox VE, which is a popular open-source hypervisor that provides an adequate API for allocation of compute resources but lacks a standard way to scale a Kubernetes cluster.

This problem is made more complex by two additional independent decisions that need to be sorted out before autoscaling can be discussed. Firstly, whether to go for a full-fledged, modular Kubernetes using Kubeadm, or a lightweight, one-binary distribution like K3s. Secondly, whether to use a completely isolated QEMU VM, or a light LXC container with a shared kernel. Both decisions have known consequences in terms of isolation, boot time, and overhead, and each may influence the ability of the cluster to scale. But what is still unclear is the joint effect of distribution and substrate on autoscaling in a private Proxmox setup.

The current research analyzes Kubernetes auto-scaling in the Proxmox VE private cloud environment using the controlled 2x2 experiment with two different combinations (two distributions, kubeadm and K3s, multiplying by two different substrates virtual machines based on Ubuntu OS and LXC containers). The key point is that all four clusters are implemented in a similar manner using only one approach for defining infrastructure as code (IaC) and therefore any difference may be attributed either to the distribution or to the substrate only.

1.1 Aim

The aim of the thesis is the development and evaluation of an effective autoscaling Kubernetes setup on a self-hosted Proxmox VE private cloud platform. Moreover, the influence of selecting different Kubernetes distributions (kubeadm vs K3s) and the compute substrate (virtual machine vs. LXC container) on autoscaling efficiency and complexity will be investigated. Specifically, cluster-level autoscaling will be examined, which is not yet widely studied outside of public environments.

1.2 Objective

In pursuit of the above-mentioned objective, this research endeavors to fulfill the following clearly defined objectives:

1. Create a replicable private cloud testbed based on Proxmox VE 9 and surrounded by an outer layer of the FreeBSD gateway system (FreeBSD/PF, HAProxy, and FRRouting).
2. Implement kubeadm and K3s both on the Ubuntu VM and the LXC containers, thus creating the matrix of four different configurations to compare.
3. Implement all four configurations through a declarative OpenTofu pipeline in which only the substrate and distribution are variables to be considered.
4. Configure Horizontal Pod Autoscaler and Cluster Autoscaler and load each configuration with reproducible, repeatable load profiles created by JMeter.
5. Gather the metrics that were recorded in this study – Time-to-Scale (TTS), Provisioning Success Rate, and Error Rate (dropped requests) via the observability stack consisting of Prometheus and Grafana.
6. Derive SWOT analysis of the distribution, substrate, and IaC approach in question, thus providing practical guidance.

1.3 Proposed Solution

This solution is based on a multi-tier private cloud model, where every tier has one job to perform, and where the four cluster topologies differ only in terms of distribution and substrate. The following sub-sections describe the different tiers of this solution. System Architecture Layers.

1.3.1 Layer 1: System Architecture Layer

The computing stack comprises three layers that interact with each other. The topmost layer includes an external FreeBSD gateway where all incoming packets are filtered: FreeBSD/PF manages the firewall edge and protects against ingress, HAProxy balances the client requests among Kubernetes worker machines, and FRRouting announces cluster routes over BGP to route the incoming traffic to the right backend despite topology changes in the cluster. Underneath the gateway layer lies the Proxmox VE 9 virtualization manager that is running directly on physical hardware and supports four clusters that are compared in this research. These two layers are complemented by a third observability/load-testing layer in which Prometheus and Grafana collect metrics from Proxmox and Kubernetes clusters respectively, whereas JMeter generates load. Figure 1 shows the interaction between the layers.

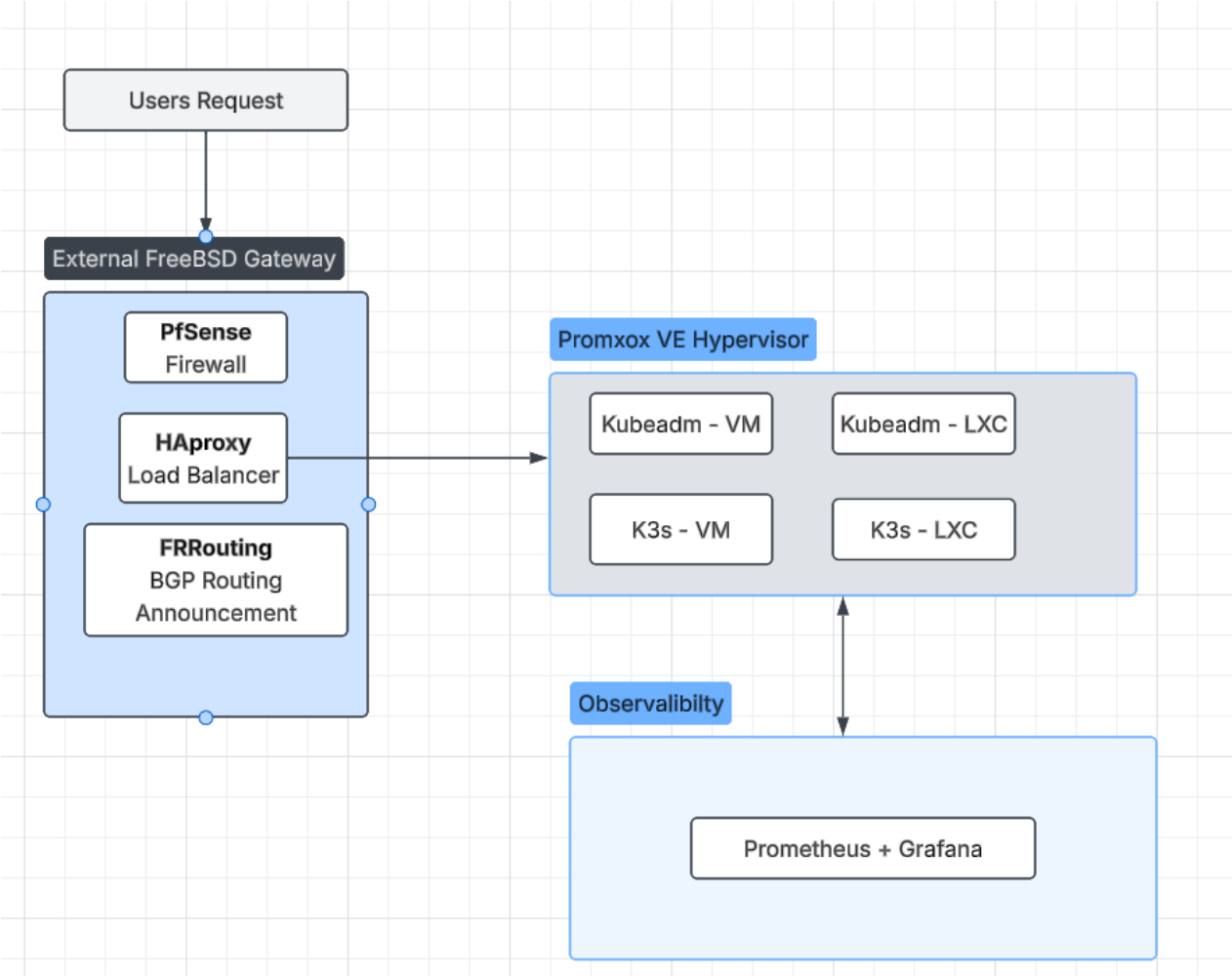


Figure 1: End-to-end system architecture across all layers

Table 1 makes the comparison matrix transparent. Each cell represents a completely autonomous scaling Kubernetes cluster. The columns change the substrate while the rows change the distribution. As such, moving across a row provides the effect of the substrate, while moving down a column provides the effect of the distribution.

	Ubuntu Virtual Machine	Ubuntu LXC Container
Kubeadm	Full Isolation, dedicated kernel, slower boot, heavier footprint.	Shared host kernel, faster boot, requires nesting and cgroups configurations. (This option, of course, comes with the risk that the use of CRI-based container cluster management in an unprivileged LXC may be subject to capability limitations as well as socket namespace constraints; this cell remains an experiment to prove a point.)
K3s	Lightweight distribution on a fully isolated VM.	Lightweight distribution with near-instant boot on a shared kernel.

Table 1: The 2×2 experimental comparison matrix

1.3.2 Declarative Infrastructure as Code

All infrastructure is set up declaratively using OpenTofu, which is an Infrastructure-as-Code software written by HashiCorp[1], with the use of its Proxmox provider. Rather than dealing with the Proxmox GUI or running imperative commands and scripts, the state of every node in terms of its template, resources, network configuration, and cloud-init setup is defined in the configuration files and then synced with Proxmox via OpenTofu[2]. When setting up a new node, OpenTofu makes a clone of the required template, and then cloud-init sets the hostname, network configuration, and SSH keys during the first boot and afterwards runs the join command[3].

The common application of OpenTofu for all four configurations is a purposeful decision. The use of one declarative way of provisioning of both virtual machine and container clusters allows comparing the substrates and distributions rather than provisioning tools. That is why the results mentioned in further chapters can be safely attributed to the variables under study.

1.3.3 Node Provisioning Tooling

Two specially designed candidates were considered before choosing OpenTofu as the provisioning layer; these are CAPMOX and Kproximate. They are both designed specifically to provision nodes for Kubernetes on Proxmox and, differently from OpenTofu, are tightly integrated with the scalability system of Kubernetes and not an external triggering component. However, the consideration of these systems shows that there is one serious limitation present in them, which makes them unsuitable for use as the uniform provisioning layer

CAPMOX

CAPMOX is a Cluster API[4] provider implementation for infrastructure by IONOS Cloud that provides support for ProxmoxCluster and ProxmoxMachine custom resources. In the case when the Kubernetes Cluster Autoscaler[5] scales up a Machine Deployment, CAPMOX reconciles the number of desired machines by cloning a VM template using the Proxmox API and populating it with a cloud-init script from the Kubeadm Bootstrap Provider (CABPK). This way, an entirely automated Kubernetes-native provisioning cycle is created without any out-of-band triggers.

This technique is highly advanced and elegant for VM-based clusters. The crucial problem with CAPMOX for this research is the fact that the ProxmoxMachine resource provided by CAPMOX can only clone virtual machines[6]. The Proxmox LXC container API offers a completely different set of primitives for interacting with the infrastructure, and there is no functionality to provision and manage LXC containers[7] available in CAPMOX. Thus, CAPMOX would be able to provide provisioning capabilities for only two VM cells of the 2×2 matrix, while the other two LXC cells will lack such a tool. Using CAPMOX selectively will create a tooling mismatch between the VM and LXC arms, confusing the substrate comparison.

Kproximate

Kproximate is an independent Kubernetes node scaler for Proxmox that periodically queries the cluster for unscheduled pods and, upon detection, picks a Proxmox host to clone an existing VM template that is pre-configured to be added to the cluster at the first boot [8]. Contrary to CAPMOX, it does not rely on the Cluster API infrastructure but serves as an all-in-one alternative to both Cluster Autoscaler and provisioner components. Its documentation and examples of templates target mainly K3s as the primary distribution, although other distributions are also mentioned as theoretically compatible in case the right template is crafted.

Kproximate faces the same limitation as CAPMOX concerning this work. Provisioning in this case is done by cloning a QEMU VM template with enabled cloud-init using the Proxmox QEMU API. LXC containers, created and managed through another set of commands (pct), are unsupported. Like CAPMOX, adoption of this software will limit automated node scaling to the VM cells of the matrix. Additionally, a potential issue is the small number of maintainers of the project, which may pose problems with its sustainability in a research environment supposed to run longer than the thesis period

OpenTofu

The disqualification condition for both alternatives is the same, neither CAPMOX nor Kproximate can deploy LXC containers. Since two out of the four matrix instances need to either use LXC or need a provisioning method which behaves uniformly on both substrates, a tool which focuses solely on the VM aspect cannot be used as a uniform experimental layer. OpenTofu, via bpg/proxmox provider, makes available both VM API and LXC Container API via a common declarative interface. It is thus the only tool among those evaluated which can deploy all four matrix instances via a common method and this is exactly what is needed from an experimental point of view to attribute the observed differences in the auto-scaling performance to the distribution and the substrate.

Criteria	CAPMOX	Kproximate	OpenTofu
Substrate Support	Virtual Machines	Virtual Machines	VMs and LXC Containers
LXC Container Support	No	No	Yes
Distribution Focus	Kubeadm	K3s	Distribution-agnostic
Autoscaler Integration	Cluster API (via CABPK)	Standalone (replace CA)	External Trigger
Management Cluster	Yes (CAPI Controller)	No	No
Maturity	Active (IONOS Cloud)	Single Maintainer	Mature Ecosystem

Table 2: Comparison of node provisioning tools evaluated for the study.

1.3.4 Autoscaling Control Loop:

Autoscaling in the suggested solution proceeds in two consecutive stages. The first stage takes place at the pod level and includes using the Horizontal Pod Autoscaler to monitor the load of the pods and to add new replicas as the load grows. If the number of pods exceeds the available capacity on the nodes and makes it impossible to schedule them, the Kubernetes scheduler sets these pods as Pending. It becomes a trigger point for the next stage: the Cluster Autoscaler identifies pending pods, recognizes that the cluster needs additional capacity, and triggers the creation of a new node through the OpenTofu pipeline. However, it is essential that the scheduling state inside Kubernetes should be linked with the declarative OpenTofu pipeline within the self-managed private cloud infrastructure, because there is no such thing as a cloud provider's API within a managed platform. The envisaged target architecture includes an event-driven middleware chain, which consists of observing unschedulable pods with Prometheus, raising a webhook with Alertmanager, validation of the Hypervisor Core Exhaustion Index (CEI) threshold, and triggering "tofu apply". However, in the implementation that has been created, this chain is intentionally simplified, because a Python watcher (pending_watcher.py) directly checks the Kubernetes API for pods in the Unschedulable state and applies the OpenTofu itself (without the Prometheus-Alertmanager-webhook middleware delay; see Chapter 5). Once the booting process is finished with cloud-init configuration and

cluster joining, those previously pending pods will be scheduled on the new node.

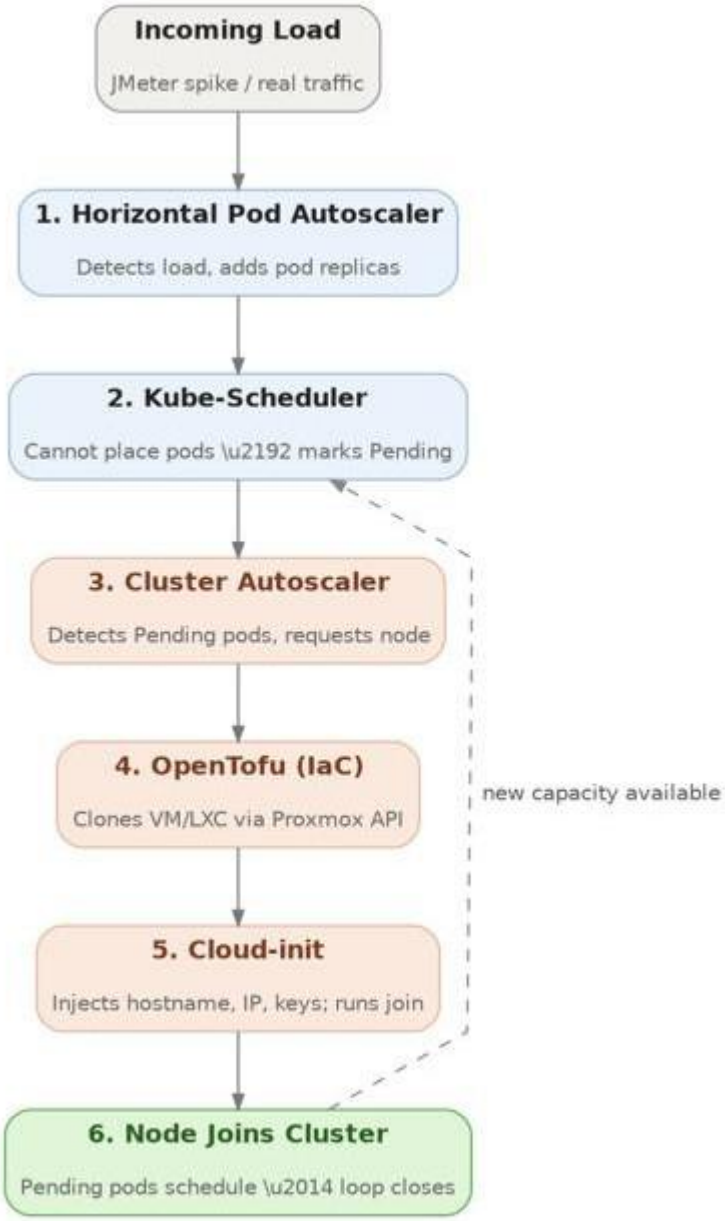


Figure 2: The pod- and cluster-level autoscaling control loop.

Though both levels are deployed and observed, the research gives the Cluster Autoscaler more weight in terms of analysis. Pod-level autoscaling is a control-plane matter does not depend much on the substrate or distribution; there's already plenty of literature on that topic. The Cluster-level autoscaling, on the other hand, is the interface that interacts with nodes bootstrapping, boot time and join protocol – the layers we expect to depend on the substrate and distribution, but is poorly analyzed when deployed on self-managed, on-premises bare metal clouds compared to the node pools offered by the public cloud vendors.

1.3.5 SWOT Analyses on Kubernetes Distributions

The first method involves analysis of the two Kubernetes distributions in question. While Kubeadm creates a fully standards compliant control plane with great granular control of all components, it has increased overheads due to its complex installation and upgrade process. On the other hand, K3s packs the entire cluster into a single light binary executable with reasonable defaults, which provides greater speed and reduced resource requirements but less flexibility.

1.3.5.1 Kubeadm

Kubeadm creates a complete Kubernetes control plane installation that conforms to the upstream design, whereby each constituent of the control plane such as the API server, scheduler, controller manager, etcd and container runtime is installed separately. In the case of Proxmox, Kubeadm is normally deployed in QEMU virtual machines, as this platform assumes a full OS environment.

Strengths	**Description**
Upstream Adherence and predictability	The use of Kubeadm builds up a standardized CNCF compliant Kubernetes cluster. Consequently, the scaling capabilities will be accepted by the reference Kubernetes scheduler and Cluster Autoscaler semantics. Thus, the chances of results being biased due to modifications on the specific distribution are minimized.
Control Over Scaling Components	It is possible to gain direct access to the control plane components of Kubernetes, thus making it possible to configure the scheduler, controllers, and the kubelet component. The approach will facilitate the optimization of autoscaling process on Proxmox VE.
Mature Node Joining Mechanism	This method of joining Kubernetes is reliable and well-documented, making it easy to integrate into Cloud Init or Infrastructure as Code processes.
High Availability	Kubeadm supports several control plane nodes and external or stacked etcd setups, enabling testing of autoscaling capabilities in high availability settings.

Wide Ecosystem Compatibility	Because of its upstream compatibility, Kubeadm is compatible with a broad range of CNI plugins, monitoring, autoscaling tools, and other Kubernetes add-ons.
Lifecycle predictability	Kubeadm offers well-defined upgrade procedures, enhancing sustainability of the experimental setup.

Weakness	**Description**
Higher Resource Consumption	Kubeadm demands more CPU, memory, and disk space than light Kubernetes because of the full Kubernetes stack and operating system requirements it demands.
Higher Deployment Duration	The VM boot up, OS initialization, Kubernetes component deployment, and node joining processes increase latency in the scale-up process.
Increased difficulty in Operations	Handling multiple Kubernetes components increases the complexity of handling configuration and increases the chance of additional failure in a automated workflow.
Complicated infrastructure as Code Integration	Automatic deployment process with Kubeadm is complicated by certificate creation, join tokens, kubelet deployment, and runtimes.
Higher Maintenance	Kubeadm deployment demands more Kubernetes knowledge for maintenance.

Opportunity	**Description**
Baseline Reference Comparison	Since kubeadm is the upstream Kubernetes system, it is a perfect base for comparison with other lightweight distributions like k3s.
Research in Autoscaling	With full access to all Kubernetes modules, it becomes possible to perform autoscaling and scheduling policies experimentation.
Practical implication in production	The results obtained via kubeadm can be applied to most enterprise Kubernetes clusters.
Optimized Provisioning	VM templates and pre-installed components can help decrease the time taken for node provisioning.

Threats	Description
Latency Issues	Long latency during scaling may affect the efficiency of the autoscaling system when dealing with rapid changes in workload.
Configuration drift	Continuous deployment might lead to configuration drifts in different experiments due to changing versions and configurations.
Maintenance Issues	The complexity of Kubeadm can be greater than the capacity of the research environment to maintain it.
Resource Usage	Greater resource usage can impact measurement in case of conducting several experiments at the same time on one Proxmox infrastructure.
Kubernetes Evolution	Evolution of Kubernetes can affect long-term reproducibility.

1.3.5.2 K3s (Lightweight Kubernetes)

K3s is a bundle that includes a complete Kubernetes cluster within a small binary with reasonable default settings, built-in container runtime, and an option for small footprint data storage system. Being light and fast, K3s is a perfect choice for running within LXC containers in Proxmox, but it is also able to operate in a VM and become part of the comparison.

Strengths	Description
Low Resource Overhead	k3s features a lightweight architecture and has lower CPU and memory needs than the standard Kubernetes installation. This enables greater node density on Proxmox VE servers and lowers the resource footprint of dynamically created nodes.
Short node provision and boot time	Due to its lightweight nature, k3s is able to start nodes very quickly. This decreases node creation time, which is one of the aspects considered when measuring the responsiveness of cluster autoscaling.
Simple node addition mechanism	The procedure of adding nodes by using the k3s agent registration is simple to configure and easily integrates into Cloud-Init and OpenTofu-based provisioning systems.
Low Complexity	k3s merges a number of Kubernetes components into one easy-to-use distribution with many default settings, thus lowering the complexity of configurations.
LXC deployment	With its low resource consumption, k3s is well-suited for use within Proxmox LXC containers.
Kubernetes certification and compatibility	Although k3s is light-weight, it remains a CNCF-certified Kubernetes distribution.

Weakness	Description
Reduced configuration flexibility	Some advanced Kubernetes configuration options are not available due to the reduced architectural capabilities of k3s.
Deviation from upstream components	k3s uses custom or bundled Kubernetes components which means some behavior is not aligned with a reference Kubernetes instance.
Reduced isolation in LXC environments	In case of running k3s in Proxmox LXC containers, it gains benefits of a shared kernel but suffers from the lower level of isolation offered by containerization technologies as opposed to QEMU VMs.
Less experience with large production setups	Even though k3s can be used for production, it is less adopted in large enterprises compared to upstream Kubernetes distributions.
Integration limitations	Default settings of k3s might cause extra integration efforts when integrating custom networking, storage or monitoring tools.
High availability limitations	k3s can be used to set up highly available systems but some deployments require extra validation when compared with Kubeadm clusters with external etcd.

Opportunity	Description
Low-latency auto-scaling evaluation	As k3s demonstrates quick start capabilities, there is an opportunity to evaluate low latency cluster scale up in a private cloud environment.
Edge and resource-constrained computing	The lightweight nature of k3s makes it relevant for testing in resource-constrained environments, thus opening possibilities for edge computing use cases.
High private-cloud density	Due to lower consumption of resources, there can be more Kubernetes nodes in the same Proxmox cluster, thus providing an opportunity to study better infrastructure utilization.
Simplified Infrastructure-as-Code process	The simplified setup procedure allows having more maintainable provisioning workflow using OpenTofu and Cloud-Init.
Experiment iteration speedup	Faster deployments and quicker recovery processes will allow conducting more experiments during the same time.

Threats	Description
Different behavior from upstream Kubernetes	Possible differences in component set might impact test results making it hard to establish the origin of the behavior.
Limited isolation capability of LXC containers	Improvements made using LXC containerization won't be applicable in environments requiring greater workload isolation.
Unconfirmed scalability capabilities	There is less proof of scalability than in the case of kubeadm about extremely large clusters.
Smaller ecosystem compared to upstream Kubernetes	Certain integrations will have lower community support than Kubernetes as a whole.
Dependency on Proxmox substrate features	Performance gains achieved by using k3s in LXC environment may greatly rely on specific kernel and cgroup settings.

1.3.5.3 Comparative Discussion: Kubeadm vs K3s

The two analyses reveal a constant tension between conformance and control, versus efficiency and speed. Kubeadm provides an entirely compliant and configurable solution with excellent high availability characteristics, but at the expense of higher footprint, slower node bootup, and increased complexity, all which impact directly on the cluster-autoscaling critical path. K3s reverses this equation, it reduces the footprint and speed up times, and simplifies the joining and deployment process, hence accelerating node-level autoscaling, but by reducing configurability down to the few options provided by the tool. The best performance of K3s is contingent upon lower isolation offered by LXC.

As a result, for experimental purposes, Kubeadm conformance makes it a better choice where compliance to the upstream behavior is important, but K3s' shorter turnaround time makes it a better fit for rapid iteration and exploration. For enterprise purposes, the maturity, high availability strategy, and isolation of Kubeadm make it a better choice. However, K3s is more appropriate for edge computing and limited infrastructures. Finally, large clusters will benefit from Kubeadm well-known scalability and rich ecosystem support, while K3s' ability to handle very large clusters is yet to be confirmed.

Given the nature of the thesis problem, the SWOT analysis not only defines the nature of each distribution but also sets up an expectation that can be tested. Given the fact that booting up nodes and provisioning are the major determinants of latency for auto-scaling clusters, the SWOT analysis indicates that K3s (especially when using LXC) will demonstrate the fastest scale-up times, whereas kubeadm (especially when using VMs) will provide the best isolation, control, and compliance. This contradiction is precisely what is expected from the 2x2 analysis, and the SWOT analysis both motivates and sets up hypotheses for

interpreting its findings. This particular risk has been identified for the Kubeadm+LXC configuration (Config-B): The CRI-based control plane comprising multiple containers and any CNI that relies on the NET_ADMIN/NET_RAW capabilities could be inherently incompatible with an unprivileged LXC substrate, an assertion whose validity is shown in Chapter 5.

1.3.6 SWOT Analysis of Virtual Machine Based and Container Based Infrastructure

This section compares the two compute substrates used to deploy the Kubernetes distributions, fully virtualized QEMU virtual machines and kernel-sharing LXC containers, both running on Proxmox VE. The reason for this is that the compute substrate defines how rapidly and thoroughly a new node is created and isolated, hence, it forms the core of any discussion about a private cloud autoscaling solution. There are two SWOT analyses provided below.

1.3.6.1 Virtual Machine Based Infrastructure (QEMU)

QEMU Virtual Machine Running on Proxmox represents a virtual machine that has its own kernel, emulated hardware, and OS environment. It is a conservative, highly isolated version of substrate and thus an appropriate host for the whole Kubeadm distribution.

Strengths	**Description**
Strong Kernel-level isolation	Every virtual machine has its own dedicated kernel, which ensures good isolation of workloads and the Proxmox host from each other. This can help to mitigate the effects of failures or security breaches when experimenting with autoscaling.
Production-like Setup	QEMU virtual machines emulate the real-life environment used in enterprises and public clouds and therefore increase external validity of experiment findings.
Compatibility with Kubernetes infrastructure	A whole operating system allows us to use the whole Kubernetes ecosystem out-of-the-box without any extra configurations.
Predictable and repeatable behavior	Full virtualization allows for stable and well-defined behavior of Proxmox VE, which can ensure repeatable automated setups in experiments.
Mature Infrastructure-as-Code support	Proxmox VM templates and Cloud-Init allow for reliable provisioning workflow, which works smoothly with OpenTofu.
Good fit for isolated workloads	High isolation properties of virtual machines make them suitable for security-sensitive and multi-tenant environments.

Weakness	Description
Longer boot times and provisioning latencies	QEMU VMs must be emulated and booted before they become part of the Kubernetes cluster, which causes increased node provisioning latency.
Higher resource overhead	The need for separate operating systems and kernels in each VM leads to higher consumption of CPU, memory, and storage space compared to containers.
Lower node density	Due to the higher resource consumption of VMs, they provide a lower consolidation ratio compared to LXC containers, leading to less elasticity on fixed hardware resources.
Longer iteration time during experiments	Creating and destroying VMs takes more time than deploying and tearing down containers, which means that it will take more time to complete the iteration process in the experiment.
Higher storage demands	The size of disk images in VMs is much bigger than in containers, which limits the number of experiments running simultaneously in the Proxmox environment.

Opportunity	Description
Golden image Optimization	By pre-configuring Kubernetes components and container runtime inside the Proxmox VM templates, we will be able to greatly cut the time required to deploy nodes but still maintain the isolation properties of virtual machines.
Production-like benchmarking	The VM substrate resembles production infrastructures very well, which means that results of experimental autoscaling can easily be applied to production settings.
Resilience and isolation experiments	Virtual isolation allows studying how failures can be contained, workloads isolated, and recovery performed when using autoscaling. This is valuable for studying the resilience of private clouds.
Benchmarking reference baseline	Being a more expensive form of virtualization, QEMU virtual machines give a good basis for comparison with the container-based substrates.
Advanced Lifecycle management	Proxmox VE offers well-developed VM management features, such as live snapshots and live migration, which could be used in the future to study more advanced forms of node lifecycle management and autoscaling.

Threats	Description
Latency in provisioning may impact service-level objectives	Given the relatively long process of booting up and initializing VMs, the nodes' availability might be delayed in the event of a sudden rise in workload. Provisioning may be unable to meet the demands of autoscaling, leading to a failure to fulfill the performance and availability expectations of applications.
Limited resources may hamper scalability	Given the increased requirements for CPU, memory, and storage of VMs, the maximum number of nodes that can be set up in the fixed Proxmox infrastructure will be lower.
Resource contention may impact results of the experiment	With VMs requiring more host resources than bare metal machines, there will be an increased competition between them, affecting results obtained from running the experiments.
Storage capacity restrictions	Given the large size of VMs, considerable storage resources will be required to store VM images. When performing large-scale experiments with a high rate of creating and deleting nodes, insufficient storage capacity may create problems.

1.3.6.2 Container Based Infrastructure (LXC)

LXC Container in Proxmox uses a shared host kernel and offers a segregated user space. This is a type of substrate that is light weight and dense and is the one on which the most responsive node scaling is possible.

Strength	Description
Immediate startup and lowered scale-up latency	Since LXC containers do not use hardware emulation and do not boot their own separate operating systems, Kubernetes nodes can achieve Ready status much quicker than VM nodes, thus lowering the scale-up latency.
Very low overhead on resources	LXC containers have no overhead from the separate booting of a new OS kernel. As such, the usage of computing and RAM per node is decreased, which increases efficiency of the infrastructure and scales the workload at a lower cost.
Higher Node Density	Containers being lighter allows a greater number of Kubernetes worker nodes to be launched on the same Proxmox server compared to QEMU VMs, giving greater elastic capacity.
Faster provisioning and destruction	Quick creation and destruction of containers lowers the duration of experiments setup and allows running more

	autoscaling scenarios and testing more workloads during the time available for research.
Good fit with lightweight Kubernetes distributions	Together with k3s, LXC is a very efficient solution that allows conducting low latency autoscaling experiments.
Less Storage Needed	LXC containers require less storage since they have light root file systems compared to full virtual machine disks used when more than one experimental setup is used concurrently.

Weakness	**Description**
Less isolation due to shared kernel	As the containers share the same kernel as that of the host, there will be less isolation between different Kubernetes nodes as compared to the virtual machines. If there is a problem in the host kernel, it can create a problem in all containers at once.
More configuration requirements for Kubernetes	Kubernetes components run inside the LXC container will need some configuration changes, including container nesting, setting up Cgroups and security, among others.
Host kernel configuration dependency	Workload running inside the containers depends on the host kernel of Proxmox, and therefore, the use of kernel-specific Kubernetes components become limited.
Network and Storage compatibility issues	CNI network plugins and storage drivers might need more configuration in LXC environment.
Lower similarity to production	Most Kubernetes environments in the production stage use VMs as the worker nodes, and therefore, results achieved from LXC environment might not be replicable in the production environment.
Workload compatibility limitations	Workloads which need special hardware support, kernel modules, and privileged access might fail to run inside LXC environment.

Opportunity	Description
Evaluation of minimum autoscaling latency	Because of its lightweight provisioning, the LXC container technology allows us to evaluate the minimum latency of nodes scaling in the Kubernetes cluster on top of the Proxmox.
Edge and resource-constrained environments	Because of its lightweight approach, LXC is an ideal choice for evaluating Kubernetes autoscaling in resource-constrained environments such as edge computing environments.
Infrastructural efficiency gain measurements	The increase in the density of nodes can help to perform an efficient comparative study on elastic capacity of Kubernetes container and virtual machine environments.
Infrastructure as Code simplifications	Because of its lightweight approach, the provision of containers can help simplify the process of using OpenTofu and Cloud-init workflows.
Experimental evaluation scalability	The faster cycle of provisioning and tearing down allows for larger scale load test experiments.

Threats	Description
Applicability limits due to isolation limitations	The less isolated nature of LXC containers may limit the applicability of such containers in secure or multi-tenancy production environments, preventing generalization of positive outcomes.
Configuration dependence and fragility	Deployments of Kubernetes based on LXC containers depend on certain nesting and cgroup configurations. Changes in the configuration of the host system can impact cluster reliability and auto scale properties.
Integration challenges influencing results accuracy	Compatibility challenges in networking and/or storage in the LXC environment might lead to unexpected outcomes, making it impossible to differentiate substrate-specific and Kubernetes-specific impacts.
Host kernel dependence	Due to using the same Proxmox host kernel, any updates or configuration changes can influence workload stability and reproducibility in the future.

1.3.6.3 Comparative Discussion: Virtual Machines VS LXC Containers

The substrate comparison brings to light the key design problem for the auto-scaling in a private cloud – speed & density vs. isolation and fidelity. The VM provides great isolation and production fidelity at the cost of start-up, overhead, and reduced density. LXC container on the other hand provides very quick start-up, low overhead and high density, making it a most reactive substrate for scale-out, at the expense of lower isolation, kernel dependency and added complexity in configuration.

For research and experimentation, LXC is better because of its faster re-building process and VM is better for production due to its isolation properties. LXC is the right choice for edge computing or limited resources environment while VM works best for enterprise production due to its isolation and maturity. For cluster computing, things are bit complicated since LXC's density makes it more suitable to pack the nodes into less hosts while VM is more predictable and compatible as scale increases.

In this thesis, the analysis results in a very specific, testable hypothesis. The substrate must have the biggest impact on scale-up latency, where LXC provisioning nodes will be fastest, whereas VMs will be the slowest but will offer better isolation and production use case, respectively. This, in turn, justifies the need for combining substrate with the distribution in the 2x2 matrix because only this way can the two factors be separated properly.

1.3.7 SWOT Analysis of the Infrastructure as Code

This part of the work presents the analysis of the declarative Infrastructure-as-Code tool called OpenTofu which was used for provisioning all nodes in this study via its Proxmox provider. It is due to a uniform IaC layer that the substrate and distribution become the sole variables of this experiment, and the strengths and weaknesses of such infrastructure layer have a significant impact on the validity of this comparison. A single SWOT analysis is offered, along with a discussion of its significance for the research.

OpenTofu represents the desired state of the Proxmox infrastructure in a declarative configuration and then reconciles it against the Proxmox API. It is being used in a uniform way throughout this research for creating all four clusters and is called upon for provisioning of nodes in the process driven by the Cluster Autoscaler.

Strengths	Description
Declarative infrastructure management	OpenTofu describes infrastructure through declarative HCL files and provides consistent reproducible provisioning of Proxmox resources in all experiments.
Support for QEMU and LXC virtualization substrate	Unlike provisioning systems that are designed specifically for Kubernetes deployment, OpenTofu supports both virtualization substrates used in this experiment, providing flexibility in all four scenarios.
Distribution-independent provisioning	Provisioning does not depend on the Kubernetes distribution, which allows us to have an identical automation approach for Kubeadm and k3s.
High reproducibility	Infrastructure is version-controlled and reusable, which guarantees identical environments in each step of our evaluation process.
Cloud-Init integration	OpenTofu has high integration capabilities with Cloud-Init, allowing for automatic provisioning of the operating system and Kubernetes node installation.
Flexibility and extensibility of automation	A modular approach allows the introduction of new infrastructure elements, such as network and storage, to the provisioning pipeline without methodological changes.

Weakness	Description
No ongoing reconciliation	For OpenTofu to work, it needs to be manually triggered and does not constantly listen to infrastructure states. Therefore, for Kubernetes Cluster Autoscaler, there needs to be an external way to trigger provisioning.
State file maintenance	State of infrastructure is stored in a state file which needs to stay consistent and protected. Corruption of the state file will interfere with automated provisioning.
Latency due to provisioning request	Using an OpenTofu apply command will introduce some extra latency between the decision of scaling and actual node creation.
Configuration drift	Manually changing configuration inside the Proxmox environment might lead to inconsistencies between the defined infrastructure and the deployed one.
Dependency on provider	Provisioning process will depend on functionality of Proxmox provider.

Opportunity	Description
Consistent experimental methodology	By using Infrastructure-as-Code technology to provision virtual machines, one can perform consistent comparisons of all Kubernetes installations because they will have equal provisioning methodology.
Reusable research platform	Since the Infrastructure-as-Code is declarative, it can be used again, and extended in the future to perform other experiments.
Node provisioning optimization	Optimization through integration with optimized VM, LXC, and Cloud-Init templates can improve node provisioning speed without loss of reproducibility.
Complete Infrastructure as Code	By using additional Proxmox components in the Infrastructure as Code solution, you can create a fully reproducible private cloud.
Methodology portability	The use of the infrastructure-as-Code technology allows one to adapt the methodology for any hypervisor/cloud platform with minor adjustments.

Threats	Description
Infrastructure state corruption	Corruption of the infrastructure state file can cause disruption of automated provisioning and lead to non-reproducibility of experimental outcomes.
Provider or API changes	The future changes of the Proxmox API and the OpenTofu provider could affect the configuration of provisioning and impact the reproducibility of the results.
External trigger failure	Since OpenTofu uses the external process for triggering provisioning, any failure in the trigger process could inhibit the node creation in case of autoscaling.
Infrastructure drift accumulation	The discrepancy between the stated and the actual infrastructure state during a prolonged experiment could affect the reproducibility of evaluation results.

1.3.7.1 Discussion: OpenTofu as an Experimental Control

As the connecting tissue of the study, OpenTofu does not offer speed, but rigor, ensuring that all configurations get provisioned in one declarative, version-controlled way means that only the substrate and distribution will vary between the matrix elements. OpenTofu's disadvantages, like its inability to reconcile, manage the state file, and the latency of its triggers, are true, but limited to specific stages of the autoscaling loop and are not relevant to the comparison.

Nonetheless, it suits perfectly well the environments of research and reproducible enterprise provisioning, where having an auditable definition is highly beneficial; and its non-reconciling approach makes it not ideal for production-level autoscaling without a triggering mechanism, which, on the contrary, is explicitly defined in the thesis. The declarative model works equally in edge, resource-constrained, and massive scales – hence it can serve as a neutral layer of provisioning in all matrix cells.

To summarize, the SWOT analysis has proven that OpenTofu acts not as an experimental variable, but rather as an experimental control since by maintaining consistency and transparency regarding the provision of autoscaling services, the thesis can attribute the performance of autoscaling services to the distribution and the substrate used.

1.3.8 Metrics Collection & Monitoring Infrastructure

Each experiment is equipped with instrumentation such that the evaluation is based on quantitative measurement rather than qualitative observation. Prometheus is employed to gather time-series metrics using its scraping capabilities and exporter configurations [9]. Grafana is utilized to visualize the gathered metrics via dashboards in order to monitor and analyze system behavior during the experiments[10]. Apache JMeter is used to create reproducible workload profiles that trigger scaling activities and measure the effectiveness of auto-scaling behavior [11]. Table 6 shows the main metrics gathered for each of the four configurations examined.

1.4 Advantages of Proposed Solution

The suggested approach is accompanied by several benefits.

- **Controlled experiment:** All four setups use the same OpenTofu pipeline, allowing one to distinguish differences caused by the distribution or the substrate from those caused by the provisioning mechanism, a solid defendable experimental setup.
- **Separation of concerns:** With the help of the layers' architecture, each concern ingress, virtualization, orchestration, and observability. It makes the system easy to extend and reproduce
- **Label the real gap:** By putting the focus on the cluster-level autoscaling in a private Proxmox substrate, the work deals with an actual gap left by the public cloud-based literature.
- **Reproducibility:** Being declaratively described and version-controlled, the test bed allows independent reproduction of results and further reuse in subsequent studies.
- **Judicious tooling selection:** Tooling selection (1.3.3) proves that the choice of the tools was not made arbitrarily but based on the comparison with custom alternatives.
- **Concrete recommendations:** Conducted SWOT analyses allow translating raw data into actionable decisions for engineers creating autoscaling Kubernetes clusters on private substrate.

2 State of the Art

2.1 Introduction

The aim of this chapter is to place the current research into the context of the existing body of literature regarding Kubernetes auto-scaling and private cloud infrastructure, thus laying the scientific groundwork for the rest of the thesis. This chapter does not simply list technologies; instead, it proceeds through the logic of a state-of-the-art review by first presenting the basic technologies which need to be known before moving on to the existing literature surrounding the issue, and concluding with a summary of how this thesis aims to fill that gap.

The literature review is structured into four major parts. In section 2.2, the basic technologies involved, namely, the container orchestration with Kubernetes, its two possible distributions, the two virtualization substrates provided by Proxmox VE, the two levels of autoscaling, and the concept of Infrastructure-as-Code, which ties everything together are presented. In section 2.3, the previous research on the topic is reviewed based on the three thematic threads that align with the three variables studied. Kubernetes distribution comparison, substrate comparison for virtualization, and autoscaling evaluation. In section 2.4, these themes are combined to specifically determine the research gap present in the existing literature concerning private cloud autoscaling, forming the foundation for the experimental design of this thesis. The focus is intentionally kept on material related to autoscaling in a self-managed, bare-metal hypervisor environment. Considering the overwhelming focus of the literature on managed public cloud environments, this aspect is stated explicitly since that is precisely what this study is about.

2.2 Background and Foundational Technologies

This section sets up the technical base upon which the surveyed literature and the current study stand. Each subsection sets out one of the pillars of the thesis and explains the terms and processes used in subsequent sections.

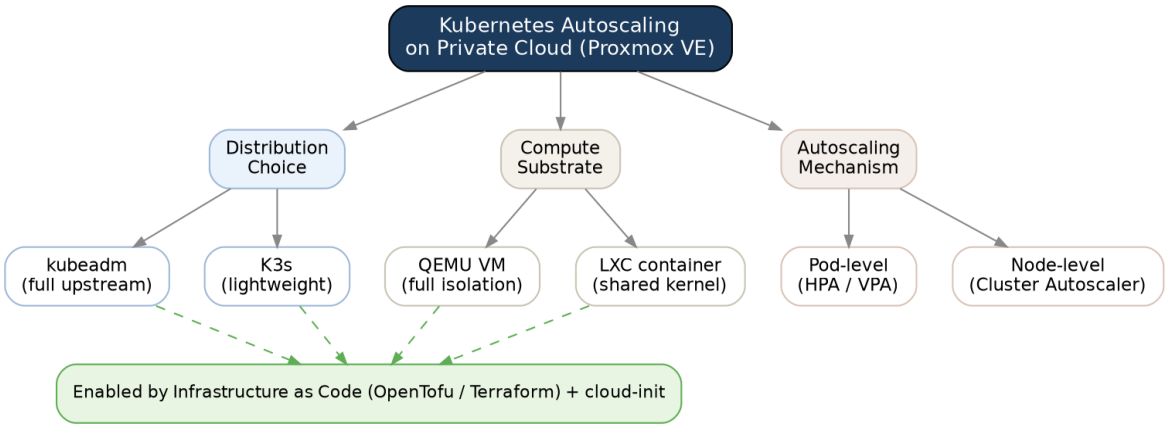


Figure 3: Taxonomy of the design space for Kubernetes autoscaling on a private Proxmox cloud.

2.2.1 Container orchestration and Kubernetes

Containerization involves packaging up the application and its dependencies into a lightweight package that uses the operating system kernel of the host system rather than virtualizing the hardware. With increasing number of containers in the system, it is essential to have an orchestration layer to schedule, network, scale, and heal them. Kubernetes, which was originally built by Google and is managed by the Cloud Native Computing Foundation today, is the default choice for the purpose. It provides an API based on a desired state that the operator needs to define, such as number of replicas of a workload, networking of workloads, and the amount of resource that they can consume.

The components of the Kubernetes system are the control plane consisting of the API server, scheduler, controller manager, and etcd database, along with a set of worker nodes where the workloads, in units known as pods, are run. It is the ability of Kubernetes to provide elasticity for variable workloads that makes it appealing, and the way this works is through a process of reconciliation, since capacity is stated declaratively and could therefore be automatically modified in reaction to the load.

2.2.2 Kubernetes distribution

Kubernetes is not an all-encompassing solution, but a set of specifications implemented by several distributions that vary with regards to their packaging, footprint, and deployment mechanisms. One of the reference solutions is Kubeadm, which creates a standards-compliant cluster by deploying and setting up each of the control plane components individually[12]. Such modularity allows fine-grained configuration and ensures full compatibility with the surrounding ecosystem, though at the expense of increased resource consumption and a more complex setup and upgrade process.

On the other hand, K3s provides a conformant cluster packed inside a tiny binary that has sane defaults, an integrated container runtime, and an optional low footprint datastore. While initially developed for edge computing and devices with limited resources, K3s sacrifices some configurability to achieve fast startup time and decreased memory footprint. Indeed, experiments have proven that K3s consumes less resources than a complete distribution while retaining conformity[13]. This is what makes a comparison between these two distributions – control and conformance vs efficiency and speed one of the main axes of this study.

2.2.3 Compute Substrate

The Proxmox Virtual Environment, the open-source hypervisor that serves as the basis for this research, provides two types of virtualization technologies, and the decision whether to cc one of these two technologies is the second factor of comparison. Virtual Machine, implemented with the help of QEMU technology, imitates the hardware entirely and runs a separate guest OS with its own kernel, but in exchange, it provides high isolation, consumes more resources, and has a lengthy boot. On the other hand, an LXC container uses the same host kernel and provides an isolated user space. As a result, this type of virtualization has low overhead, native speed, and fast booting [14].

The primary comparison between the two types of virtualizations was made by Felter et al. Their highly cited paper showed that, for most cases, container virtualization outperforms or is at least comparable to hypervisor virtualization, with main overheads incurred in I/O-bound workloads. Further studies have validated this point for a range of different workloads and at scale [15]. Since the time needed to create and start up a new node lies right on the critical path of autoscaling, the substrate difference is expected to be one of the key factors affecting the responsiveness of autoscaling. A claim backed by the literature but not yet verified in this context.

2.2.4 Autoscaling Mechanism

Kubernetes allows for scaling in two different levels which run one after another. The first is a pod-level Autoscaler, where the Horizontal Pod Autoscaler (HPA) watches the workload metrics like CPU and memory usage and changes the number of pod replicas. This HPA can scale up and down only within the capability of the existing nodes – when it asks for more replicas, than the current nodes cannot host, then the scheduler is not able to allocate it, and it becomes in a state of Pending. This situation causes the second level of scaling - Cluster Autoscaler (CA) which identifies pending pods, concludes that more capacity is needed and creates a new node. The third scaling method is Vertical Pod Autoscaler which changes resource requirements of the existing pods and not the number of them, and it falls out of the scope of this research.

An important architectural component that differentiates public cloud from private cloud autoscaling is the existence, in managed platforms like GKE, EKS, or AKS, of a provider-specific plug-in for the Cluster Autoscaler that allows the scale-up decision to be converted into a node pool API call supported by the provider [16]. For a self-managed virtual machine like Proxmox, there is no such plug-in, and the link that needs to be established between the decision to scale up and the creation of an active and joined node must be designed by the developer.

2.2.5 Deeper Look at Kubernetes Architecture

A Kubernetes cluster is a distributed system comprising of a control plane that stores the desired state and facilitates reconciliation and several worker nodes that run the workloads. The control plane has four components. The most important component at the bottom level is etcd, a strongly consistent distributed key-value store, which records the full state of the cluster – every object, every specification and every status field in the form of serialized data using the Raft consensus protocol. This means that since etcd commits writes to the disk via quorum and fsync, it not only serves as the single point of truth, but also one of the major causes of latency and I/O load for the control plane. The only component allowed to interact with etcd is kube-apiserver, which provides a RESTful API, with all requests passing through authentication, authorisation (RBAC) and admission control layers. The kube-scheduler monitors the pods without node affinity and associates every such pod with a node based on a two-stage process of filtering (feasibility) and scoring (preferences). In the case where there is no suitable node satisfying the resource requirements of the pods, the pods are set in the Pending state, which is exactly what leads to the triggering of the cluster-wide auto-scaling operation.

Each worker node consists of three different pieces. The kubelet is the node agent; it performs node registration, consumes pod spec, and controls its lifecycle via the Container Runtime Interface (CRI) executing liveness and readiness probes. The kube-proxy sets up the kernel networking stack (either iptables or IPVS) to achieve the Service concept, while container runtime (containerd, for example) starts containers.

This is the reason for the “weight” difference central to this dissertation. In Kubeadm, each control plane piece Api server, scheduler, controller manager and etcd itself – becomes a distinct container managed by CRI, thus control plane becomes multi-container construction, where each piece has its own memory footprint and bootstrap procedure (certificate generation, static pod manifests, tokens exchange). On the other hand, K3s merges all these components into a single binary running as separate goroutines inside a single process, as well as replaces etcd with an embedded database (SQLite, accessed via kine). This

results in a significantly lower footprint and a shorter bootstrap critical path that constitutes the difference in latencies measured in Chapter 4.

2.2.6 The Mathematics of Autoscaling

There are two levels of Kubernetes autoscaling which are separated. One is called a Horizontal Pod Autoscaler (HPA). It is a proportional controller which recomputes the desired replica count based on the observed metric at regular intervals. The formula for HPA control looks like:

- $desiredReplicas = \lceil currentReplicas \times \frac{currentMetricValue}{desiredMetricValue} \rceil$

- $|\frac{currentMetricValue}{desiredMetricValue} - 1| \leq \tau$

where "currentReplicas" is the number of pods in Ready state, "currentMetricValue" is the value of the metric calculated from those pods' values (when talking about CPU – average utilization in proportion to requests for CPU), and "desiredMetricValue" is some value chosen by the operator. The ceiling function $\lceil \cdot \rceil$ guarantees that the result is an integer and tends to oversize. To prevent flapping (oscillation) when the value of the metric changes, HPA uses the tolerance zone τ (default = 0.1):

The node that maintains U "node" less than a defined threshold value (0.5 by default) within a given grace period will be drained and subsequently shut down, if the pods can be scheduled on other nodes. In this dissertation, the function of the scale up of CA will be achieved through the custom watcher, and node creation will also depend upon the CEI value (Section 1.3.4).

$$U_{\text{node}} = \frac{\sum_{p \in P_{\text{node}}} \text{requested}\,(p)}{\text{allocatable(node)}}$$

2.2.7 Operating System Internals: Virtual Machines vs. Containers

These two different compute substrates diverge in the placement of the isolation boundary itself. With a QEMU/KVM VM, one achieves full hardware-level virtualization where each VM guest will have a completely independent operating system running on emulated hardware (or para-virtualized virtio). Classically, there are four protection levels available on the x86 architecture: only Rings 0 (kernel) and 3 (user) are typically utilized. In a guest OS, there are privileged operations that a kernel needs to perform which can no longer be executed directly in Ring 0 on the host. The solution comes from hardware-level assistance of the virtualization (Intel VT-x/AMD-V technology) which introduces an orthogonal notion of

root and non-root modes of operation where the guest kernel can run in Ring 0 mode but with non-root permission set which will make certain sensitive instructions to VMEXIT to the hypervisor.

The LXC container implements OS-level virtualization where all containers use the host kernel, and isolation is constructed by combining two features of the kernel. Linux namespaces, namely PID, NET, MNT, UTS, IPC, USER and cgroup, confine the system resources visible to the container, while control groups (cgroups) impose resource limitations on CPU, memory, and I/O. Since there is no second bootstrapped kernel, start-up performance is close to native – the source of the low latency of the LXC substrate.

At the same time, the shared kernel feature is the reason why Config-B fails in Chapter 5. An unprivileged LXC uses a user namespace where the container's root (UID 0) is mapped to an unprivileged UID on the host, and Linux capabilities are restricted to this namespace. While a process can have NET_ADMIN (interface, routing, and firewall management) and NET_RAW (packet sockets) capabilities in its user namespace, they do not apply to resources in the host's initial network namespace. As a result, privileged socket operations that are performed at initialization time by all mainstream CNI plugins (Flannel, Calico, Cilium) are failing with EPERM, regardless of the overlay used.

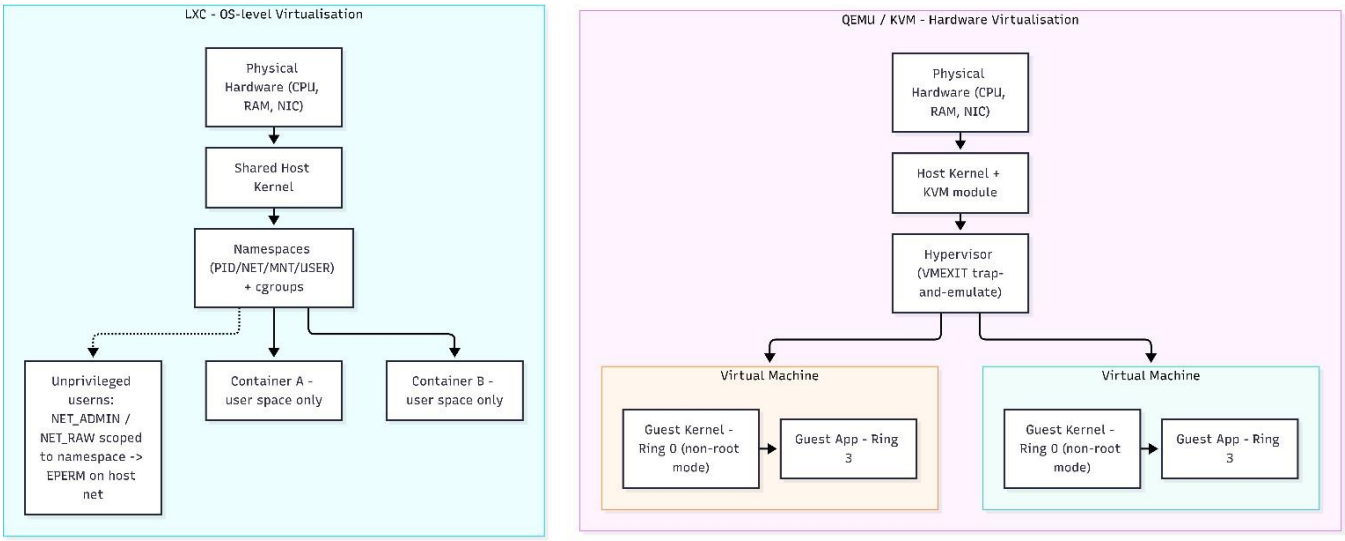


Figure 4: Comparison between Architectural Design of QEMU (Hardware Virtualization) & LXC (Operating System Level Virtualization)

2.3 Existing Research

This review explores the literature related to the variables in the thesis statement, which can be broadly classified into three major streams: comparison of Kubernetes distribution, comparison of virtualization substrates, and assessment of autoscaling methods. Each stream will be studied for the conclusions drawn from it and, importantly, for the limit beyond which its conclusions cease to be valid for the private cloud.

2.3.1 Comparison of Kubernetes Distribution

Recently, many studies have been done on lightweight and full versions of Kubernetes mainly due to the popularity of edge computing. Koziolek and Eskandani performed an extensive comparison of MicroK8s, K3s, K0s, and MicroShift to provide reproducible benchmarking results and proved significant distinctions between lightweight distributions concerning startup times and resource usage [13]. The surveys of empirical studies of lightweight Kubernetes for edge computing always demonstrate that K3s has the lowest resource usage while full Kubernetes is better concerning throughput in the data plane and ecosystem.

Relevant in this context is the study where the comparison of Kubeadm, K3s, MicroK8s, and K0s occurs at the same time as varying the virtualization technology and the container run-time[17]. Such an approach in this research stands out from others because of how both the distribution and the underlying technology are treated as varying factors instead of one being fixed when studying the other. In parallel, studies have compared the distributions based on their ability to ensure security compliance, resilience in network partitioning situations, and maintainability, and they have discovered that due to the simplicity that makes K3s efficient, it is less secure [18]. However, all these studies have examined the distributions in steady-state situations, unlike the one in the current thesis.

2.3.2 Comparison of Virtualization Substrates

The comparison between hypervisor and container virtualization techniques is one of the well-established subjects in computer science literature. The pioneering work by Felter et al. has shown that Linux Containers reach near-native performance level and are on par or superior to virtual machines, with overhead being mostly I/O bound [19]. In their work, Xavier et al. have further explored the performance trade-offs between containers and hypervisors focusing on such technologies as LXC around high-performance computing, where they measured trade-offs between performance near native level of containers and the strong isolation provided by hypervisors [14].

Sharma et al. studied containers and VMs at scale, presenting the notion of hard vs soft, overcommit able resource constraints associated with the two and showing their direct effect on node density in a shared host system [15]. More recent studies have broken down container booting into its steps to identify methods to pinpoint exactly the source of the provisioning delay. This line of research proves beyond doubt that containers are faster to boot, have higher density, but offer less isolation than VMs. The question that has remained unanswered so far is how this disparity translates up to the operation of the Kubernetes cluster Autoscaler, i.e., how it affects autoscaling latency.

2.3.3 Evaluation of Kubernetes Autoscaling

Kubernetes scaling research is divided based on whether it occurs at the pod/node interface. Studies that focus on pods include the research by Nguyen et al., which presents an extensive experimental study of the horizontal pod Autoscaler and discusses its scaling trend, cluster size, metric scraping period, and latency, in addition to the implications of the use of default resource metrics versus Prometheus metrics [20]. Recent studies have sought to improve pod-level autoscaling with respect to parameter tuning, predictive scaling, and machine learning approaches.

Node level autoscaling has not been studied as much as pod-level autoscaling has been. The closest contribution in the literature would be by Tamiru et al., who conducted an experimental evaluation of Kubernetes Cluster Autoscaler and noted explicitly that although there has been lots of work done on the topic of cloud autoscaling, autoscaling on Kubernetes has not seen any work [16]. Importantly, their evaluation just like any other study on cluster-Autoscaler is performed on a managed public cloud platform, which means that the node provisioning functionality is owned by the provider of the cloud service. Other attempts on improving cluster-autoscaling include the control theoretic and optimization-based methods to improve node scalability and predictive approaches that attempt to predict the demand to provision resources in advance and thus reduce scale-up time. In all those studies, the node provisioning substrate is assumed to be an opaque entity provided by the cloud provider.

2.4 Research Gap and Justification

The comparative study reveals an exact gap among three well-known research areas. The following is a clear articulation of the gap and the reason for its existence.

The gap may be formulated in the following way. The behavior of Kubernetes cluster autoscaling within a self-hosted, bare-metal private cloud and in particular, the influence of the joint decision regarding the Kubernetes distribution and compute substrate, has never been thoroughly researched. Public-cloud research ignores the process of provisioning which is explicitly required in a private cloud [16] distribution and substrate research provide the description of their subject separately and in steady state [13], [19] finally, the Infrastructure-as-Code layer used to map scaling decisions into node instantiation on

a hypervisor has not been investigated within the context of the scaling loop [1]. Each area is quite developed in itself; their intersection is almost absent.

Several factors necessitate the bridging of this gap. Firstly, self-managed private clouds based on the use of hypervisors like Proxmox are becoming increasingly prevalent due to considerations of cost savings and data sovereignty, but there is no autoscaling capability available in the manner found in public clouds; therefore, advice in this scenario would be useful. Secondly, since there appears to be an interaction between the choice of distribution and substrate with respect to autoscaling responsiveness, i.e., light-weight distributions and shared kernel containers both offer quicker provisioning, the choices that engineers must make in terms of the design cannot be influenced by existing literature.

Consequently, the research problem is answered in a 2x2 experimental design, which cross-matches two different Kubernetes distributions (Kubeadm and K3s) with two different compute substrates (i.e., Ubuntu VMs and LXC containers) within a single Proxmox VE private cloud, where the uniform provisioning of clusters is carried out via OpenTofu and evaluated both on the pod and cluster level with the latter being of specific interest. The use of the same provisioning mechanism but of two different Kubernetes distributions and substrates enables identifying the impact of each factor on the efficiency of autoscaling and its complexity, thus providing exactly the sort of joint evidence that the reviewed literature lacks.

3 System Specification and Requirements

3.1 Project Overview

This chapter formulates the specification that derives from the motivations and directions specified in the previous chapters. In the same way that Chapter 1 outlined the goal of the thesis and Chapter 2 discussed the context of the work within the state of the art, this chapter outlines precisely what the system must do (functional requirements), what it must achieve (non-functional requirements), and how its behavior is going to be evaluated (monitoring and evaluation specification). The requirements are systematically formulated from the goal of the thesis as seen in Figure 5.

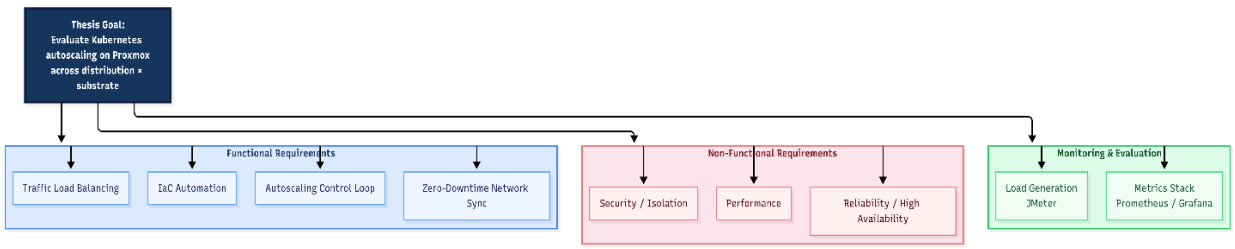


Figure 5: Derivation of the System requirements

3.1.1 Problem Statement Validation

In Chapter 1, the problem this thesis is focusing on has been identified, and in Chapter 2, it has been proved by a literature review: There is no standard mechanism for mapping Kubernetes' scaling decisions onto compute node provisioning in a private cloud where the infrastructure runs on a Proxmox VE cluster, and neither the effect of the Kubernetes version nor the underlying compute platform has been systematically investigated [13], [16].

For the specification to be well-formed, the problem itself must be established as real, non-trivial, and measurable. It is real since the private cloud providers are truly deprived of the turnkey autoscaling, whereas public-cloud providers enjoy turnkey autoscaling and do not have to build the provisioning bridge themselves. It is non-trivial since the bridge covers several interacting aspects declarative provisioning, node bootstrapping, cluster joining, and route synchronization, all of which can potentially fail separately and be influenced by the underlying system and infrastructure under investigation. It is measurable since the aspect of interest how fast and reliable a scale-up request will transform into a serving node can be measured via clear metrics like scale-up latency and node boot time. Therefore, the verification of the problem statement directly implies the subsequent requirements

3.1.2 Scope and Boundaries

The scope of the research is deliberately limited to ensure the study remains controlled and the results obtained are interpretable. These aspects are listed in Table 4.

In Scope	Out of Scope
• Two Kubernetes distributions-Kubeadm and K3s • Two compute substrates for Proxmox VE – Ubuntu QEMU VMs and LXC Containers • The corresponding 2x2 matrix consisting of four functional autoscaling clusters • Pod-level autoscaling as a controlled precondition, and cluster level autoscaling as study object. • Declarative provision of all four clusters using OpenTofu and bpg/Proxmox provider • External FreeBSD gateway and in cluster MetalLB and BGP-based route synchronization • Observability using Prometheus and Grafana, and reproducible load generation using JMeter	• Pod Autoscaler and other resource-right-sizing techniques. • Autoscaler for Proxmox (CAPMOX, Kproximate) that have been excluded as they would fail to provision the LXC containers and disrupt provisioning uniformity within the matrix. • Multi-hypervisor or multi-data center infrastructure; in the scope of the research there is only one cluster of Proxmox VE. • Tuning application-level efficiency of the workloads used for workload generation; workloads are tools, not the object of research. • Security hardening beyond isolation capabilities provided by the two underlying substrates.

Table 3: Scope and Boundaries of the study

3.2 Functional Requirements

The functional requirement states the capabilities the system must have. Each of them is written with an identifier, description, and verification criteria that would determine whether it has been achieved. There are four functional domains, and they represent the cooperating concerns of the provisioning-and-serving bridge from Section 3.1.1.

3.2.1 Infrastructure as Code (IaC) Automation

It is required to provision each node of each configuration in a declarative manner without user intervention to have clusters instantiated, terminated, and re-created reliably in many experiments. This is accomplished by means of OpenTofu with bpg/proxmox as the provider reconciling desired and actual states via Proxmox API. To create a node, a template is cloned, then cloud-init supplies the hostname, network settings, and credentials on the first boot, and the join command specific for the node's operating system is executed. The identical procedure is required for both VMs and LXCs, because provisioning consistency is the characteristic which makes the substrate comparison meaningful.

3.2.2 Autoscaling Control Loop

The autoscaling system should provide for a closed autoscaling loop, working on two levels. At the pod level, Horizontal Pod Autoscaler should add pod replicas because of load, while if there is not enough capacity on the current nodes to run new replicas, the scheduler should flag the pods as Pending. At the node level, the Cluster Autoscaler should detect pending pods and understand that more capacity is needed; then it should activate the IaC layer and create a new node. When the node boots up, joins the cluster and is registered, pending pods should be rescheduled. This loop should work in all four setups, and its behavior in terms of timing, especially the latency of the node-level part, will be the main measurement point for the evaluation.

3.2.3 Zero-Downtime Network Synchronization

Since the cluster configuration evolves due to autoscaling operations, the networking layer needs to catch up without disrupting the services. This means that the network routing state has to be synchronized automatically in case of the node addition or removal operations. This is ensured using MetalLB running in the BGP mode and peering with the external FRRouting running on FreeBSD-based gateway machine advertising routes for LoadBalancer type of services. As seen in Figure 6 below, when Cluster Autoscaler creates a new node, MetalLB advertises the routes to FRRouting over an existing BGP connection so that the traffic can start flowing to the newly created capacity as soon as possible without any manual configurations and disruption of connections. This is the description of the target architecture, since the FreeBSD/FRRouting edge could not be provisioned in the project (Chapter 5), the zero-downtime requirement was fulfilled in the implemented architecture using MetalLB Layer 2 (ARP) failover in

addition to kube-proxy, resulting in the exact end-to-end feature of zero dropped connections when scaling up.

3.2.4 Traffic Load Balancer

The architecture should allocate incoming traffic from clients to available worker nodes so that extra capacity can be utilized, and no node gets overloaded. The distribution of incoming requests on the FreeBSD gateway by HAProxy is done at layer 4 and 7 among the currently available worker nodes. Along with route synchronization in Section 3.2.3, this helps in ensuring that when a new worker node is brought into the cluster through autoscaling, it starts getting an equal amount of traffic right away.

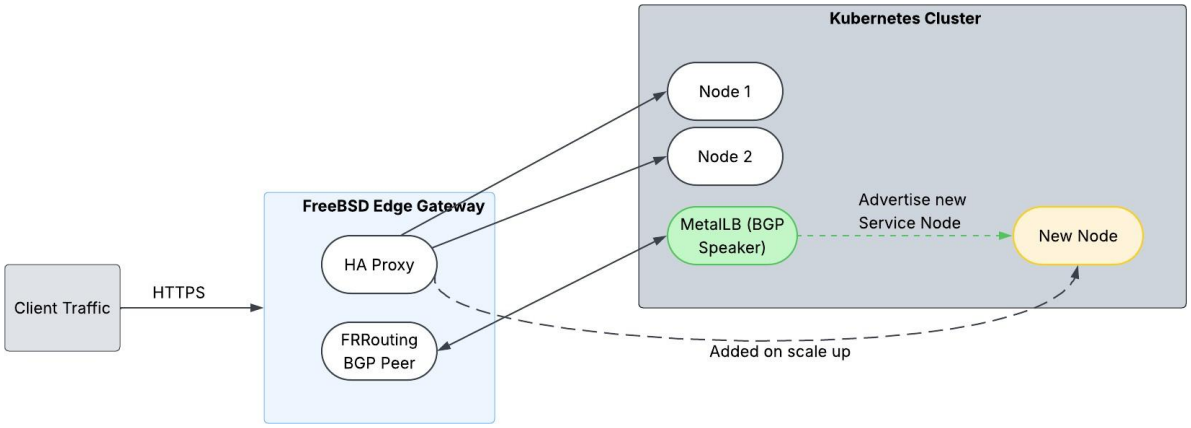


Figure 6: Zero-downtime network synchronization

ID	Functional Requirement	Verification Criterion
FR-1	**Declarative Infrastructure Provisioning:** The system should use OpenTofu and Cloud-Init for provisioning Kubernetes control plane and worker nodes on both Ubuntu VM and Proxmox LXC platforms.	Provisioning running tofu applies the same identical operational state to the cluster without any human intervention, making it idempotent across all infrastructures.
FR-2	**Hierarchical Control Loop for Scaling:** The system should be capable of performing a hierarchical scaling process whereby HPA performs pod replication during high loads that will invoke the cluster Autoscaler to provision additional physical node capacity via the Proxmox REST API.	On observing 80% CPU usage through the HPA, a "pending" state of the pods is triggered, which gets provisioned with a new node, and the "pending" state of the pods changes to a "scheduled" state in all four test setups.
FR-3	**Zero Downtime:** Dynamic Routing Table Synchronization: The design should incorporate newly provisioned nodes into the routing table via BGP peering of the MetalLB speaker daemon and the FreeBSD FRRouting daemon.	Zero TCP connection losses occur in the scale-up process, and the proxy daemon configuration does not need to be restarted and reloaded during scale-up.
FR-4	**Edge Level Traffic Load Balancing:** The edge level FreeBSD gateway should leverage HAProxy/PF to distribute HTTP requests among the compute nodes.	The externally simulated workload via Apache JMeter is visibly and evenly spread across all worker nodes, including new provisioned nodes as well.

Table 4: Functional Requirement and their verification criteria

3.3 Non-Functional Requirement

The non-functional requirements of the system determine the properties which the system should demonstrate in the process of performing its functions. This being a comparative study, some of these properties become the dependent variables to be studied and are thus identified as objectives of the system as well as measurable values.

3.3.1 Performance

The performance becomes the crucial non-functional factor to examine since the responsiveness of the autoscaling becomes the focus of research. The system must provision and integrate the new node fast enough to become useful in realistic conditions and its performance needs to be assessed uniformly in different configurations. The crucial performance indicators include the scale-up latency (time span from the first Pending pod to scheduling of that pod), node boot time (from clone request to Ready node) and pod scheduling delay. Since the node boot time represents one of the critical path components and differs dramatically between LXC substrate that boots quickly and VM substrate which boots slowly, the mentioned factors are defined as comparative parameters and not threshold ones since they need to be measured consistently for all configurations.

3.3.2 Reliability and High Availability

Scalability needs to happen not only quickly, but reliably there is no point in scaling up quickly if the process fails regularly. Scalability reliability is defined as a ratio of successful scale-ups while under load, or the ratio of scaling decisions that produce a properly connected and working node without any additional action required. The availability of the control plane must be guaranteed throughout the process of scaling up, since scalability decisions must keep being correct while the system is stressed out. Whenever this is possible, the highly available setup of the control plane is desirable, since its reliability is a factor influencing whether autoscaling continues while being under load.

3.3.3 Security and Isolation

There should be a proper isolation of workloads, and importantly from a standpoint of this paper, the difference between the isolation provided by the two substrates should be recognized as a first-class feature and not as a secondary one. Virtual machines offer high-level kernel level isolation, while LXC containers have a shared kernel environment and hence offer less isolation and have a bigger blast radius due to the kernel level errors. Additionally, nesting is needed along with proper Cgroup and security configuration when Kubernetes runs inside LXC. In the periphery a FreeBSD/PF firewall is required for protecting the inbound traffic at the gateway The goal is not to achieve maximum security, it does not fall into the scope of this study but to preserve the isolation features offered by both substrates.

ID	Non-Functional Requirement	Verification Criterion
NFR-1	Performance and Autoscaling	Responsiveness Measured by evaluation metrics quantitatively through the concurrent workload tests simulated with Apache JMeter. The system should precisely evaluate and compare the time-to-scale (TTS) for all four combinations in the 2x2 experimental matrix.
NFR-2	Infrastructural Reliability and High Availability.	The Kubernetes control plane (Kubeadm and K3s) must always remain available in a stressed condition. The system should attain the perfect success rate (100%) in dynamic node provisioning without reaching the Hypervisor Core Exhaustion Index (CEI) threshold.
NFR-3	Perimeter Security and Substrate Isolation	Maintains a clear separation of concerns through the layer 1 FreeBSD edge gateway that relies on the built-in Packet Filter (PF) for the perimeter security purposes. Moreover, the system should faithfully reproduce isolation at the kernel level and OS level as required by the isolation capabilities of the Ubuntu VM and Proxmox LXC compute substrate.

Table 5: Non-Functional requirements and how each is specified

3.4 Monitoring and evaluation Specifications

To substantiate the claims of this thesis, the measuring and evaluating framework will have to be laid out with as much rigor as the actual system. This chapter explains the data gathering framework, the measures being taken, and the experiment being conducted to compare the four systems.

3.4.1 Data Collection Stack

The metrics are gathered by Prometheus, which scrapes data from both the Proxmox hypervisor and the Kubernetes clusters and presents it in Grafana. Reproducible load, which is created by JMeter and will lead to spikes in traffic and, thus, the triggering of the tested autoscaling loops, is generated. This makes the test a closed system where JMeter perturbs the system, triggers the autoscaling loop, and then Prometheus measures its reaction, providing a fully traced experiment each time. The stack configuration remains the same for all tests, ensuring that none of the differences in results is caused by instrumentation.

3.4.2 Metrics Captured

The metrics that have been considered for each configuration have been tabulated in Table 6 below, along with the nature of each metric and its relevance to the research. The empirical measures collected in this study are Time to Scale (TTS), Provisioning Success Rate, and Error Rate (failed requests). Those measures that rely on architectural elements do not present in the final realization of the design (BGP Network Convergence Time, in particular) are omitted, and Time to Scale is an end-to-end latency metric rather than a pure node boot time.

Metrics	What it Captures	Relevance
Time to Scale	The time between the arrival of the first pod in a "Pending" state and the transition of the node into a "Ready" state.	A key metric for cluster autoscaling response times. Measuring this time will help distinguish the faster booting LXC substrate from the VM substrate.
Error Rate (Dropped Requests)	Number of client requests that result in failure during a live scale-up process, as captured in the JMeter aggregate report (client-side view).	Proves empirically that elastic scaling does not decrease availability at the connection level; distinguishes between failures and high tail latencies due to CPU saturation.
Provisioning Success Rate	Percentage of independently scalable instances that prove to be successful in creating a connected server node without any human assistance.	Difference between reliable infrastructure automation and unreliable scaling process under high Apache JMeter load.

Table 6: Metrics Captured and their Operational Significance

3.4.3 Evaluation Method

All four configurations are tested under the exact same set of JMeter load tests, each one carried out as repetitive and similar load tests to determine the repeatability of the test results as opposed to single instances; thus, the experiment describes the empirical repeatability of the test results instead of using a statistical approach. All tests are timed from the point at which the load is introduced up until the cluster scales up and stabilizes. As all four configurations are identical with respect to all elements except for distribution and substrate, the difference in the resulting metrics will be due to the two variables. Comparing across substrates shows the impact of virtualization technologies on autoscaling performance, whereas comparing across distributions shows the impact of the Kubernetes distribution itself, thus meeting the goal set in Chapter 1.

4 System Design

The architectural plan of the system, for which the requirements are laid out in Chapter 3, is described in this chapter. The design chapter deals with the design aspects of the system, not its implementation. This chapter defines the structure of the system, the division of labor within it, as well as control and routing mechanisms that allow one to satisfy functional and non-functional requirements but leaves everything related to the actual implementation of these structures (source code, providers, commands) to Chapter 5. The architectural design will be presented in a top-down manner starting from two layers overall structure of the system and moving to the experimental design.

4.1 Architectural Blueprint and Network Topology

At its very core, the system is built using an architectural separation of the layer doing the computing from the layer controlling the network ingress. This section details that separation and follows the path of the request in the resulting topology.

4.1.1 The Two-Layer Substrate Model

The system is architected using two separate planes. The compute plane is implemented using the Proxmox VE hypervisor running on bare metal and hosting the Kubernetes clusters and all worker nodes running the applications. The network ingress plane is implemented using a dedicated FreeBSD-based gateway which routes all traffic from/to the external network to/from the compute plane. The two planes are built as distinct layers having one well-defined interface between them rather than as a unified environment with routing, filtering, and computing all together.

This decoupling of layers is a key design principle and not an accidental choice. It is driven by three reasons that become particularly important in the context of a bare-metal private cloud. Firstly, it ensures a security boundary. With perimeter defense centralized in a fortified gateway that is architecturally distinct from the compute substrate, the exposed attack surface of the infrastructure due to external traffic is limited to just one dedicated layer, and hypervisor on which the clusters run is never accessible from the untrusted network directly. Secondly, it results in efficient routing and independence. As the edge takes care of all ingress traffic and dynamic routing tasks, compute plane can scale up or down thanks to autoscaling without necessitating any manual network reconfiguration, provided the two layers have dynamic routing agreement in place. Thirdly, it allows for the two layers to evolve independently, and that is useful when conducting a comparative study with four different variations of the compute plane being kept constant.

Decoupling makes the experiment valid. Because the network edge is a constant element common to all the four configurations in the comparative matrix, the difference in the autoscaling characteristics could then be due to the compute plane and the substrate rather than the edge itself. This makes the two-layer design both a good engineering practice and a prerequisite for conducting a controlled comparison.

4.1.2 Network Topology Flow

The lifecycle of an external request reveals the way these two levels interact. An HTTP request arriving from the Internet first hits the perimeter firewall on the FreeBSD gateway, where a stateful packet filter decides based on certain criteria as to whether the connection will be allowed through. Once the traffic is allowed through the firewall, it is passed to the reverse proxy and load balancing layer, which selects an active worker node from the available list of workers and sends the request to it.

In the compute plane, the request reaches the worker node selected for processing, and the Kubernetes service layer routes the request to a pod that can serve the request. The response follows the reverse route to the gateway before reaching the client. More important, scaling out by adding new nodes to the cluster does not require any manual effort to include the new node in the routing of requests. The moment the node becomes a part of the cluster and announces its presence to the edge, the load balancer starts routing traffic to it.

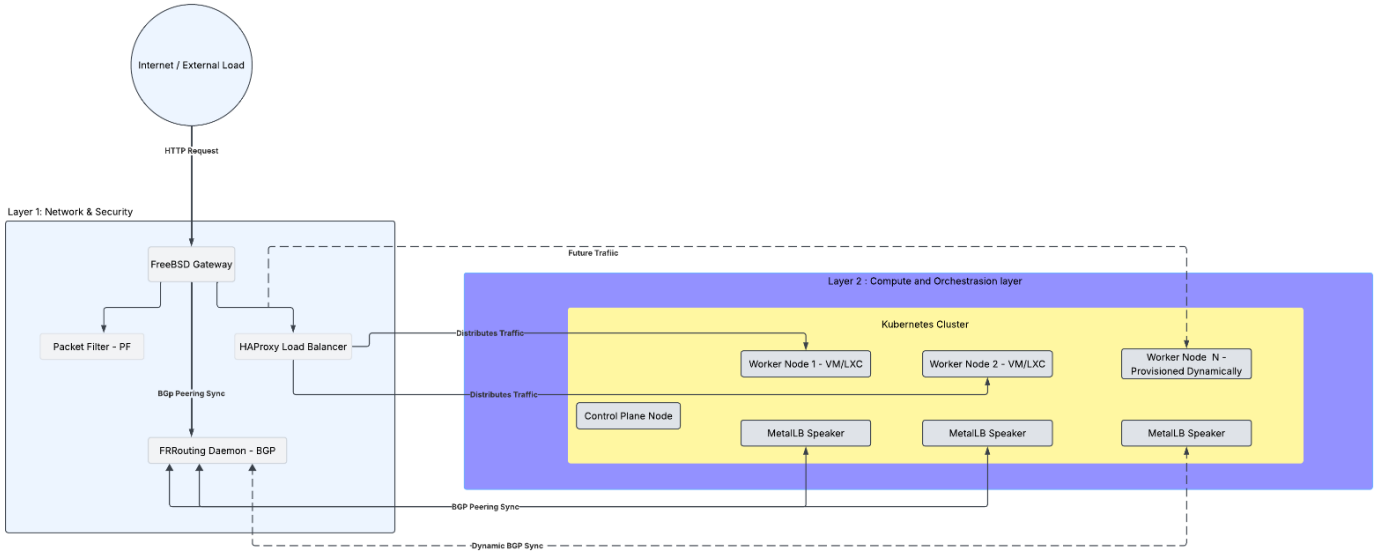


Figure 7: Complete Network Topology Architecture

4.2 Edge Routing and Security Design

On the foundations of the two-tier architecture, this chapter outlines the design of the network edge, including the perimeter-defense scheme, the dynamic routing algorithm to keep the edge in sync with the elastic cluster, and the load-balancing scheme to distribute traffic among worker machines.

4.2.1 Perimeter Defense

Perimeter Defense will use the native FreeBSD Packet Filter (PF) firewall, which will form a stateful security boundary that will protect the compute layer from external threats. The reason for choosing PF as a firewall in Layer 1 comes down to its stateful ness and architecture. Being a stateful firewall, PF tracks the states of all allowed connections and allows only those packets which were sent as a response to already existing legitimate connections, thereby preventing any unsolicited traffic from entering the compute layer. Such approach is the most secure one when it comes to protecting a hypervisor from being exposed.

Given that PF runs on the dedicated FreeBSD gateway and not the compute substrate, there is a clear physical and logical separation between the filtering boundary and the applications being protected, a compromise on any application pod cannot disable or circumvent the perimeter boundary, and the management interfaces of the hypervisor would also not be accessible from the untrusted network as per the design of the architecture. It is the intention of this architecture that the compute plane trusts the edge with filtering out the hostile traffic.

4.2.2 Dynamic Route Synchronization Strategy

The key problem for the edge to solve is synchronization between the network and the evolving shape of the compute plane in the context of autoscaling. The proposed solution is dynamic routing using the Border Gateway Protocol (BGP), implemented through the peering between the MetalLB speaker service running on the Kubernetes worker nodes and the FRRouting service running on the FreeBSD gateway. Instead of specifying the set of accessible services in static form, which will have to be configured manually upon each addition/removal of a node, this design considers accessibility as routing information.

According to this approach, MetalLB takes the responsibility of advertising routes related to Load Balancer type services and sets up a BGP session with FRRouting through which this information is communicated. If a new node is provisioned using the Cluster Autoscaler, and some workloads are scheduled onto the node, MetalLB will advertise these routes to FRRouting, which will update the routing table of the gateway with them. The benefit is the ability to have zero downtime integration, where newly provisioned resources become reachable as soon as they are ready, no disruption to any connections, and no manual intervention from an operator is needed.

4.2.3 Load Balancing Architecture

With the above routing approach, the load-balancing solution aims to evenly balance the incoming traffic across all the live worker nodes, so that new capacity can be effectively used. HAProxy, sitting on top of the FreeBSD gateway, balances the simulation user traffic across the live nodes. The load balancer uses the information about the set of live nodes, which is provided via the link to the dynamic-routing module of FRRouting.

There are two reasons why active and even balancing of traffic across the worker nodes should be employed. First, without such balancing, one of the worker nodes may become a bottleneck, and thus obscure the scaling results, which we want to achieve. Secondly, active load balancing allows to start working immediately after a new node is provisioned into the cluster. Combining the above load balancing solution with the routing policy from Section 4.2.2, we eliminate the gap between provisioning and exploitation of a node.

Traffic Type	Source	Destination Component	Protocol / Port	Security Policy
Inbound Traffic	External client (internet)	HAProxy	HTTPS / TCP 443	Stateful, allow TLS-terminated at edge
Reverse-proxy	HAProxy (edge)	Worker node service	HTTP / TCP 80	Allowed to healthy backends only
BGP route	MetalLB speaker	FRRouting	BGP / TCP 179	Restricted to peer addresses
API	Cluster Autoscaler / IaC	Proxmox REST API	HTTPS / TCP 8006	Authenticated, internal management network
Clusters	New node	Control-plane API server	HTTPS / TCP 6443	Token-authenticated. internal only
Management	Operator	FreeBSD gateway / nodes	SSH / TCP 22	Key-based, internal management network only

Table 7: Edge gateway traffic.

4.3 Autoscaling Control Loop

The autoscaling control loop is the key part where things happen dynamically. This section presents the logic behind the elasticity and the process of provision in terms of timing.

4.3.1 Elasticity Logic and Threshold

The control loop is based on two levels of the threshold-based system. On the level of pods, the Horizontal Pod Autoscaler (HPA) monitors workload utilization and triggers action if the average CPU utilizations go beyond an 80% threshold; at this moment the HPA increases the number of replicas of the pod [5]. The threshold value of 80% is set deliberately; it is sufficiently high to avoid scaling because of noise but also leaves enough space for absorbing load while providing new capacity.

When the HPA asks for more replicas to be added than what is possible by the available nodes, the scheduler cannot add those replicas, and the corresponding pods will remain in the Pending state. The Pending state is the trigger that sets off the second phase of the loop: the Cluster Autoscaler realizes that the cluster does not have enough capacity to schedule the pending workload, which indicates the need for adding more physical nodes.

One of the key aspects of the elasticity approach is the use of the throttling mechanism limiting unlimited scaling. As there is only a certain number of cores in the Proxmox host underlying our private cloud, there is a threshold for scaling in the compute plane which could be seen as a Hypervisor Core Exhaustion Index (CEI). This index could be defined as a ratio of the number of virtual CPUs provided by Autoscaler to the actual number of cores available on the host. The algorithm of the decision process is designed in a way so that the decisions regarding scaling up that lead to overcommitment of the hypervisor are throttled to ensure the performance of the workloads is not compromised. Thus, CEI works as a safety guard that helps to keep the elasticity behavior within the boundaries of the physical resources available in the private cloud.

4.3.2 Provisioning Sequence

The Provisioning Process is an explanation, from a chronological perspective, of the conversion of a scaling decision made at the Kubernetes level into a new physical node joining the cluster. This starts off with the Cluster Autoscaler detecting Pending Pods and verifying that the CEI Limit allows for scaling, after which it sends out a request to the infrastructure-as-Code layer to create more capacity. In turn, the infrastructure-as-Code layer uses the Proxmox control plane API to clone a pre-prepared template, VM or LXC, into a new node.

After the creation of the instance, its identity and connectivity are provided through a first boot configuration process and, consequently, the command corresponding to that distribution that attaches the node to the existing cluster is executed. After successful joining, the node will register with the control plane and enter the state of readiness. Then, the service routes of the node are advertised to the edge by the routing layer, and with the availability of the resource, the previously Pending pods are scheduled on the new node. This completes the loop, whereby the load that initiated the scaling is satisfied by the capacity that the scaling operation created.

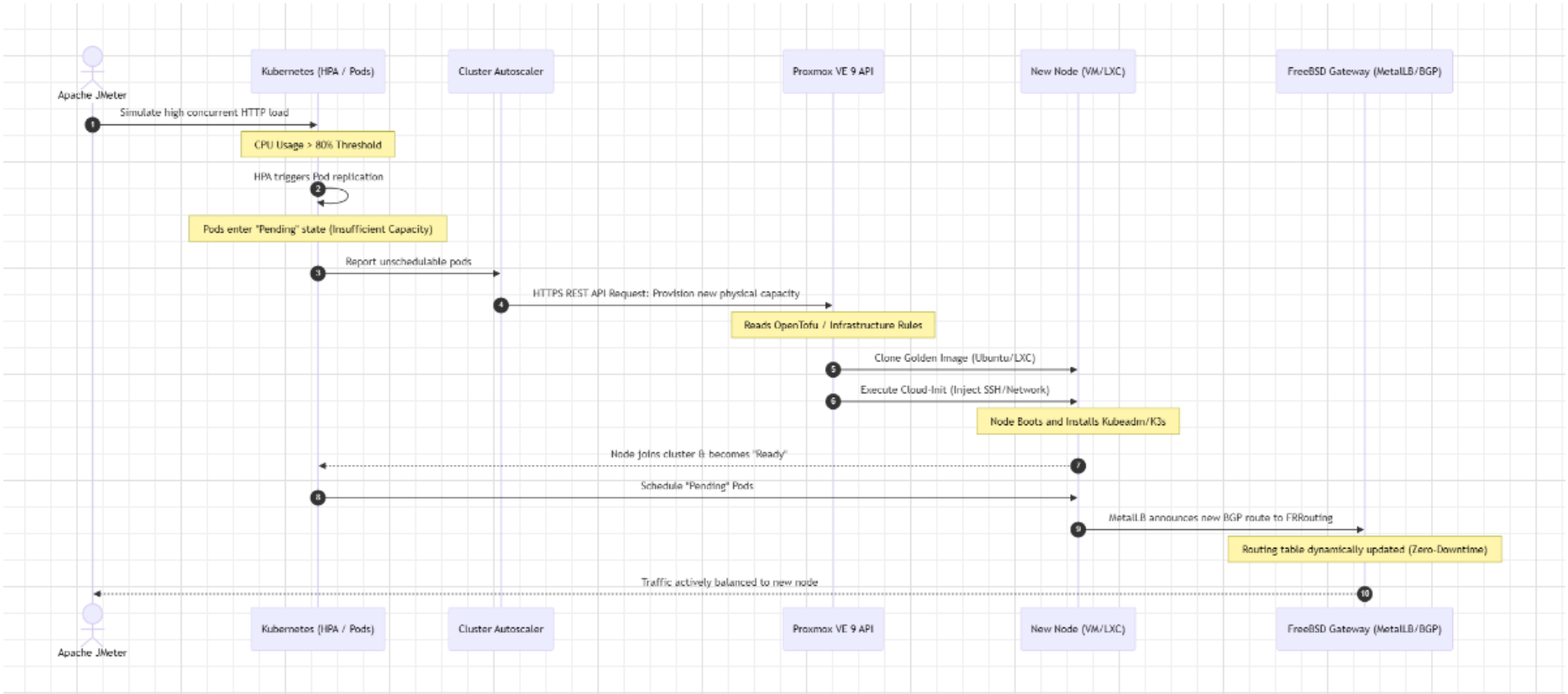


Figure 8: Autoscaling control-loop sequence

4.4 Experimental design: 2 x 2 Matrix

The architecture discussed in the previous paragraphs was designed for the purpose of enabling a controlled experiment. This section outlines the experimental framework that the architecture is designed to facilitate, namely the 2×2 configuration matrix and the hardware baseline.

4.4.1 Definitions of the Test Configurations

The comparative approach used in this research is based on a 2×2 factorial test design consisting of two orchestration frameworks paired with two compute platforms. In this case, the two orchestration frameworks are at opposite ends of the spectrum of Kubernetes distributions, namely, the fully featured upstream Kubeadm framework, which offers the entire control plane and configurability, and the compact K3s framework, which packs the entire cluster into one small executable file [7]. As for the two compute platforms, each of them embodies a different approach to virtualization. Namely, the hardware-based virtual machines running Ubuntu in this case have a dedicated kernel while the OS-level Proxmox LXC containers share the host's kernel.

Crossing these two factors gives four different configurations of the test, each being an independent autoscaling cluster. The reasoning behind the complete crossing instead of the individual testing of the two factors lies in the ability to separate the effects through the manipulation of one factor at a time and keeping the other one constant, which allows us to distinguish between the effect of the orchestration platform itself and the effect of the compute substrate, as well as their interaction effect. If we were to fix Kubeadm to VMs and K3s to LXC, respectively, the configuration and substrate would be conflated in the process. Table 8 shows four configurations along with the boot-speed profile they produce.

Config ID	Orchestration Platform	Compute Substrate	Isolation Level	Expected Boot-speed profile
Config-A	Kubeadm	Ubuntu VM	Kernel level	Slowest among all- full OS boot + Multi step Process
Config-B	Kubeadm	LXC Container	OS level	Moderate, fast Boot, multi setup
Config-C	K3s	Ubuntu VM	Kernel level	Moderate, full OS Boot, Single binary join
Config-D	K3s	LXC container	OS-level	Fastest among all + single binary join

Table 8: The 2 x 2 configuration matrix.

4.4.2 Hardware and environmental Constraint

The conditions for empirical validity of the experimental results, as well as scientific reproducibility of the experiment, include explicit physical constraints of the experiment itself. As all four configurations are implemented on the identical physical Proxmox VE infrastructure provided by Hof University, there will be no hardware advantage of one configuration over another. It is the baseline of physical constraints (number of physical CPU cores, amount of system RAM, storage subsystem, and network throughput of the host) that determines the physical constraints within which autoscaling is taking place, including the ceiling of Hypervisor Core Exhaustion Index.

It will help in two ways. First, it will make sure that the research is replicable, because the next researcher will be able to duplicate the exact environment using the specifications that have been provided and second, it will make sure that it is internally valid, because using the same hardware for all four different configurations means that any difference must come from the orchestration and the compute substrate itself and not the different hardware. The experiment design will assume a single cluster topology on a trusted internal network after the edge gateway, which means that the only perimeter, as described in Section 4.2.1, is the boundary between the experiment and the outside network. Table 9 below captures the hardware and environment baseline.

Parameter	Baseline Specification
Hypervisor platform	Proxmox VE 9 (bare metal), Hof University
Physical CPU	Intel(R) CPU E5-2620 v4 @ 2.10GHz – 2 Sockets * 8 cores = 16 physical cores, 32 Threads
System memory (RAM)	188.8 GiB (202.7 GB)
Storage subsystem	Local-lvm, 2.30 TiB total / 2.23 TiB available on (Dell PERC H730 Mini RAID controller)
CEI ceiling (derived)	0.75 (MAX_NODES=6 × 2 vCPU /16 physical cores)
Topology assumption	Single flat /22 bridge (vmbr0, 192.168.67.0/22) on the shared StudiCloud host.

Table 9: Hardware and environmental baseline

4.4.3 Result of Empirical Study 2 * 2 Matrix

The controlled 2×2 matrix was run on the physical Proxmox VE host presented above. The Time-to-Scale (TTS), which is the period between detection of the first unschedulable pod and the report that the new node is ready, was measured for each possible cell using successive identical JMeter load tests. Table 10 presents the average TTS for each configuration and identifies the two key factors which the study aimed to distinguish: the compute substrate factor (VM vs. LXC, with distribution held constant) and the distribution factor (Kubeadm vs. K3s, with substrate held constant).

Configuration	Distribution × Substrate	Mean TTS	Δ vs Baseline	Interpretation
Config-D	K3s + LXC	56.9 s	(baseline)	Fastest cell; single-process control-plane bootstrap on a shared-kernel substrate.
Config-C	K3s + VM	79.2 s	+39% vs Config-D	Isolates the substrate effect at fixed distribution the "VM substrate tax".
Config-A	Kubeadm + VM	125.5 s	+58% vs Config-C (+120% vs Config-D)	Isolates the distribution effect at fixed substrate the "distribution tax".
Config-B	Kubeadm + LXC	Infeasible	N/A	Documented as an architectural incompatibility finding (Chapter 5).

Table 10: 2×2 matrix measured Time-to-Scale results.

The basis for Config-D is the fastest configuration possible. Going from the static distribution (K3s) to adding a full QEMU VM substrate result in an increase of TTS of about 39% that includes the cost of booting an individual kernel and the cloud-init first boot process. Going from the static substrate (VM) to adding Kubeadm in place of K3s results in an increase of about 58% TTS costs associated with the bootstrapping process of a multi-container, CRI-managed control plane vs a single K3s process. The combination (Config-A vs. Config-D) of those two increases TTS by approximately 120%. The Config-B was not possible due to the limitations of the substrate; its analysis can be found in Chapter 5.



Figure 9: Grafana Dashboard: Node Scaling, Pending Pods, CPU Usage, and Network Input/Output during Scale Up.



Table 10 — underlying values

CONFIG	DISTRIBUTION	SUBSTRATE	RUN 1	RUN 2	MEAN	Δ VS CONFIG-D
Config-D	K3s	LXC	51.8s	61.9s	**56.85s**	comparator
Config-C	K3s	VM	83.4s	74.9s	**79.15s**	+22.3s · +39.2%
Config-A	kubeadm	VM	126.9s	124.0s	**125.45s**	+68.6s · +120.7%
Config-B	kubeadm	LXC	–	–	infeasible	n/a

Figure 10: Experimental Time to Scale (TTS) Comparison for Various Matrix Configurations.

4.5 Mechanics of Implementation

The above sections have described the system as a design. This section describes how this design was implemented by describing the actual provisioning code, the special-purpose control loop middleware, and the network mechanics behind this design. The following artefacts are described here as an implementation of the design; the engineering problems that occurred in creating them are discussed separately in Chapter 5.

4.5.1 Implementation of Infrastructure as Code

All compute nodes are created declaratively using OpenTofu with the community bpg/proxmox provider, which reconciles the desired-state specification described in the Hashicorp Configuration Language (HCL) against the Proxmox REST API. Instead of manually installing a node using the ISO on each scaling event, provisioning is simplified to a clone of a pre-prepared golden master node which is essentially an Ubuntu cloud image with the container runtime and all required distribution binaries installed to avoid the process of installation of packages during the boot procedure. The initialization step is the entry point for cloud-init, which configures the node with a static IP address and the user_account with SSH public key of the operator on the first boot. A sample worker configuration is presented below.

```
resource "proxmox_virtual_environment_vm" "worker" {
  count     = var.worker_count
  name      = "config-d-w${count.index}"
  node_name = var.pve_node                # bare-metal host, e.g. "s09"
  vm_id     = var.vmid_base + count.index

  clone {
    vm_id = var.template_vmid             # golden cloud-init template
    full = true
  }

  agent  { enabled = false }              # guest-agent absent on cloud image
  cpu    { cores = 2 }
  memory { dedicated = 2048 }

  initialization {
    ip_config {
      ipv4 {
        address = "${var.node_ip_base}.${var.node_ip_offset + count.index}/22"
        gateway = var.gateway_ip
      }
    }
    user_account {
      username = "ubuntu"
      keys     = [trimspace(file(var.ssh_public_key))]
    }
  }

  lifecycle {
    ignore_changes = [initialization]      # write-only fields never re-read
  }
}
```

Two mechanics specific to this provider can be observed. The first is the ignore changes = [initialization], which ensures that a false positive “forces replacement” diff does not result in the destruction and recreation of live nodes: some initialization parameters are write-only, so they are never returned in a read call to the Proxmox API. The second is the fact that there is no way to upload arbitrary cloud-init data through the Proxmox REST API, hence the join-cluster phase cannot be done as a NoCloud user-data script but must be run using the remote-exec provisioner, which will open an SSH connection to the new static IP address.

4.5.2 Custom Watcher Middleware

In a managed cloud, the Cluster Autoscaler invokes a provider plugin; in the case of this private cloud, there is no such plugin, and the custom middleware pending_watcher.py provides the necessary bridge functionality. It is one single control loop that matches the pressure of Kubernetes scheduling with the capacity limitations of the underlying hypervisor. On each cycle, it fetches from the Kubernetes API pods which carry Unschedulable condition in their status; the pods being the direct evidence that the scaling of pods has reached its limit given the current pool of nodes. If there are such pods, the watcher counts the number of Ready workers minus one control-plane-labeled node and calculates the scale up target limited with hard cap of six nodes.

The provisioning step is constrained with Hypervisor Core Exhaustion Index (CEI), that is the ratio of allocated vCPUs to physical cores on the host, ensuring the elastic layer does not over commit the bare metal infrastructure:

$$\mathrm{CEI} = \frac{\sum_i \mathrm{vCPU}_i}{C_{\mathrm{phys}}}, \qquad \text{scale-up permitted} \Leftrightarrow \mathrm{CEI}_{\mathrm{post}} \leq \mathrm{CEI}_{\mathrm{ceiling}}$$

The watcher employs the provisioning layer only if the future post-scale CEI falls within the ceiling limit. The hard-wired invocation is provided below; observe that the ANSI-stripping and destroy-guard were necessary (Chapter 5).

```
def scale_to(target):
    plan = subprocess.run(
        ["tofu", "plan", f"-var=worker_count={target}", "-no-color"],
        capture_output=True, text=True)
    clean = re.sub(r"\x1b\[[0-9;]*[a-zA-Z]", "", plan.stdout)   # strip stray ANSI
    if "will be destroyed" in clean:
        raise RuntimeError("aborting: plan would destroy a live node")
    subprocess.run(
        ["tofu", "apply", "-auto-approve", f"-var=worker_count={target}"],
        check=True)

while True:
    if unschedulable_pods() and (t := min(ready_workers() + 1, MAX_NODES)) >
ready_workers():
        if cei(t) <= CEI_CEILING:
            scale_to(t)
    time.sleep(POLL_INTERVAL)                                    # 5 s
```

The variable used in OpenTofu is worker_count. Mistakes in using the wrong variable name or counting the control plane node as a worker were the two mistakes that were rectified during implementation.

4.5.3 MetalLB Layer 2 ARP Operations

In a shared broadcast medium, the destination IP address needs to be resolved to a MAC address using the Address Resolution Protocol (ARP). The host will then send out a broadcast of "who has this IP?" and cache the MAC in its response. By leveraging this step of resolution, MetalLB in Layer 2 mode allows a service VIP to be reached without having to configure the router. In each LoadBalancer VIP case, a unique leader speaker is elected by the MetalLB using its memberlist membership protocol to answer the ARP request with the MAC address of its own network interface. All the incoming traffic for that VIP is then directed to one node, after which the distribution is made possible by kube-proxy to pod endpoints on all the nodes under the default externalTrafficPolicy: Cluster. This ensures that once the worker nodes scale up, the endpoints can be reached through the VIP owner without changing the ARP resolution.

The zero-downtime feature is realized during failover. In case of draining or failure of the leader node, the new leader quickly sends a gratuitous ARP broadcast to reassign the VIP to the MAC address of the new leader node. Since this packet is unsolicited, all hosts on the segment and CAM table in the switch are instantly updated, ensuring redirection of the VIP traffic to the new node even before the TCP connections time out. This explains how the solution met FR-3 zero-dropped-connection requirement (Chapter 5). Although the solution is not multipath equal-cost load balancing like what the initially designed BGP edge would provide, the structure limitation of the solution having a single node taking care of the incoming traffic of a VIP is the sacrifice made for the impracticality of the BGP solution.

5 Implementation Complexity & Findings

The previous chapters set out the theoretical architecture aimed for and the controlled experiment that architecture was intended to enable. This chapter describes the engineering challenges faced during the actual implementation of that architecture in the physical environment of shared, bare-metal infrastructure. Some of these challenges are not mere implementation details; rather, they are genuine research results explaining why a cell of the matrix is impossible, why the network architecture had to be redesigned, and why a particular latency figure is not equivalent to the restoration of service quality. Every challenge has been recorded dispassionately, together with the reason why it occurred whenever such a reason has been identified.

5.1 Substrate and Distribution Incompatibility: The Config-B Finding

The most important conclusion drawn from the implementation process is that Config-B (kubeadm using an unprivileged Proxmox LXC container) is not only inconvenient but fundamentally impossible under

these conditions. This conclusion came after diagnosis and not before because it is based on two completely different errors caused by the same factor: the limitation of capabilities and namespaces imposed by the unprivileged LXC.

The first problem is related to the control plane itself. To create the control plane, kubeadm starts all control plane components (API server, scheduler and controller manager) as a nested CRI-managed container. As an unprivileged LXC cannot bind listening sockets for this nested container, we managed to reproduce the problem using a simple listener running on port 6443 with host networking enabled. The restriction is indeed caused by the kernel and not by Kubernetes. The fact that K3s is not affected by this problem is explained how it works – all components of the control plane run as goroutines inside one normal process. Moving the control plane into a virtual machine, that has its own kernel, eliminates the first error.

This second problem arises only after having mitigated the first one. Now that the control plane has been moved to the VM, a Kubeadm worker in a non-privileged LXC joins the cluster, registering the kubelet as ready and starting the kube-proxy; however, none of the CNI plugins can start. In particular, the Flannel pod fails at its first attempt when trying to query its own pod spec from the API server; in fact, it returns a permission denied error during an apparently normal socket call, before applying the overlay networking logic. On the other hand, the kube-proxy, that lives on the same worker, manages to perform the exact same call to the API server without any issues. The only real difference is that the CNI pods request the NET_ADMIN and NET_RAW capabilities, necessary for network manipulation; possessing those capabilities within a nested, non-privileged LXC prevents normal socket calls from being performed. Since the problem is not CNI-backend-dependent, switching CNI backends does not solve anything, since every main-stream CNI plugin has the same capability requirements.

This means that the result is not merely a specific issue but that the combination of unprivileged LXC with any multi-container control plane managed by the CRI, as well as with any CNI needing capabilities, will never work. Config-A (Kubeadm running in a VM) remains untouched because a VM will have its own, independent kernel. This means that the concern raised in the Kubeadm + LXC SWOT analysis from Chapter 1 is proven true, turning this failure into an engineering insight about the boundaries of the design space.

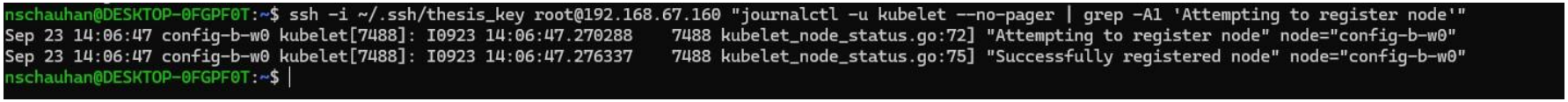


Figure 11: LXC Worker Kubelet Registration Logs (config-b-w0) Successfully Registered with Hosts.

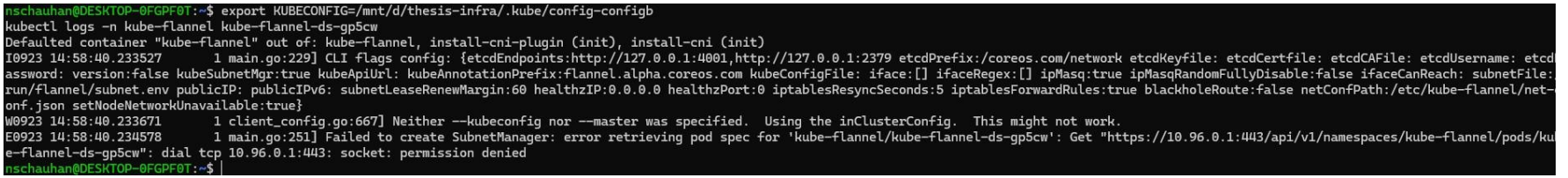


Figure 12: Nested Flannel CNI Container Log Indicating Socket Permission Error in Unprivileged LXC Environment.

5.2 Architectural Deviations and the Network Pivot

The target architecture comprised of a FreeBSD edge gateway running FRRouting, HAProxy and the PF firewall, and MetalLB connected to it in BGP mode. The edge gateway virtual machine was imported and booted successfully, yet failed to initialize the network interface, not generating any response to an ARP request and timing out SSH connections throughout two different, fully executed provisioning cycles. The problem that stood in the way of diagnosing the issue was a hypervisor-level constraint rather than any software issues. Inasmuch as the official cloud images do not come with the guest agent installed, and it was not possible to bake it into the system from the host shell either, the agent was turned off in all the project VMs, meaning that boot and cloud-init logs of the guest could not be viewed. The attempt to connect to the console via the hypervisor WebSocket console protocol resulted in a successful protocol handshake, followed by a closure of the connection, without providing any further information in the documentation outside of the vendor's own client. This deviation is therefore recorded honestly as unresolved a plausible-cause hypothesis exists (a NoCloud seed or virtio-net mismatch specific to the guest operating system) but was not confirmed in explicit contrast to the Config-B finding, whose cause was fully proven.

The approach adopted here was not a concession to the requirement but a rethinking of how it could be met. In the absence of a functioning BGP peer, the MetalLB setup was made using ARP mode with traffic distribution carried out via kube-proxy across the cluster. Full load was then applied through the virtual IP address of the service: JMeter oversaturated the single worker, and the pods were made pending. Then the watcher provisioned a second worker for use, with the load distributed to the endpoints across both workers. This successfully met the objective of the FR-3: during the entire scale-up process under sustained load, there were no failures at the connection level according to the JMeter aggregate report; any failed connection was only a client-side timeout due to tail latencies when the CPUs were saturated. An early test using a naive single-threaded probe with a very short timeout period mistakenly indicated total failure, but this was only a measure of the problem with the probe itself, not an actual network problem.

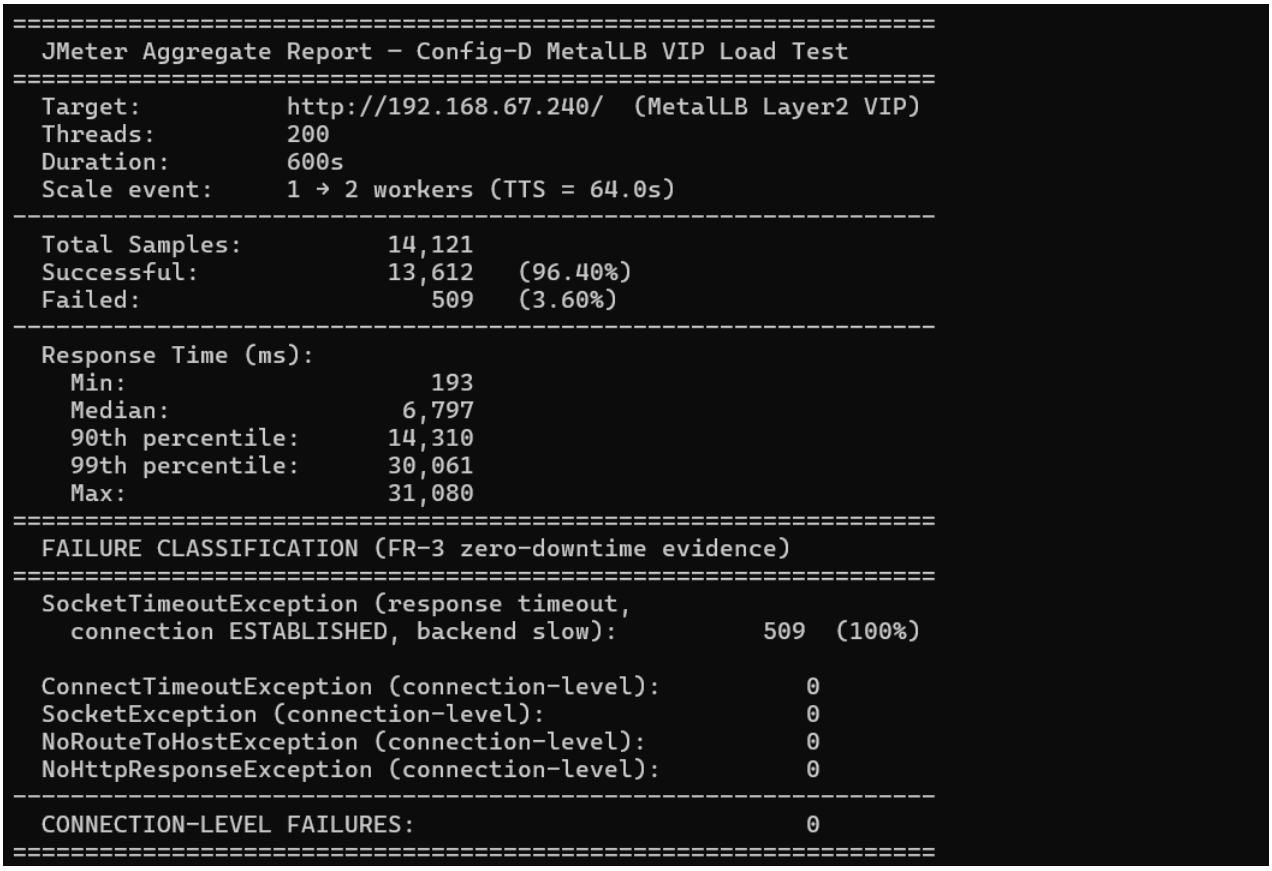


Figure 13: Aggregate Report of Apache JMeter for Config-D Showing No Connection Failures.

5.3 Time-to-Scale versus Time-to-Recovery: The Latency Recovery Lag

A notable and reportable difference is that TTS is not the same as TTR. TTS is defined as the point at which the new capacity becomes Ready; TTS is not defined by the moment when quality of service is restored to pre-scale levels. By default, Kubernetes scheduler assigns new pods to the new node; however, it does not redistribute the pods that are currently running on the saturated node. Thus, a newly added node will take the load only as long as more replicas are scheduled on it through Horizontal Pod Autoscaler, creating a multi-minute delay between TTS and full recovery of latency observed by clients. This delay is a property of the default behavior of the scheduler, not the substrate or distribution, and hence should be reported separately from TTS. Practical significance of this finding and further direction of research is described in Chapter 6.



Figure 14: Grafana Per Node CPU Utilization Showing Latency Recovery Lag Between TTS and TTR.

5.4 Measurement Validity: The Observer Effect

Given the experimental setup on limited and shared hardware resources (worker nodes with 2 vCPU and 2 GB), the observability stack affected the observed system itself. The comparison of experiments under the load of the experiment with and without the whole Prometheus/Grafana stack revealed that the overhead of the stack itself increased the average time-to-server failure and maximal error rate by a factor of two to four due to the competition for resources between HPA replicas and the LXC clone I/O. For this reason, the TTS averages presented in Table 10 are workload-only benchmarks, and the overhead of the monitoring stack is recognized as an important caveat of the experiment itself.

5.5 Selected Engineering Edge-Cases

Two additional edge cases have been included because both have explanatory worth beyond just fixing the issue. The first case involved a readiness probe deadlock. With the forced CPU saturation done for triggering the scaling behavior, the HTTP-based readiness probe that was testing the throttled application may fail for all replicas at once. The Horizontal Pod Autoscaler does not include metrics coming from NotReady pods, so the absence of a proper CPU metric due to all replicas being NotReady caused a deadlocking of the scaling up behavior. The solution was changing to using TCP-socket readiness probes since they only involve a completion of the TCP handshake by the kernel, not a response from the throttled application. The second problem involves a lack of certain functionality within the Proxmox API, which cannot upload cloud-init snippets via its REST API. Thus, the original design idea involving declarative cloud-config bootstrapping had to be changed to use only cloud-init via SSH key together with

imperative bootstrapping scripts executed with remote exec provisioners. This is a legitimate implementation complexity issue that describes the difference between the ideal and reality, not a shortcut.

5.6 Storage Exhaustion and Observability Churn

One of the challenges that arose during the extensive testing process included disk-pressure events on LXC worker nodes, which occurred more frequently than expected. The shared root filesystem with a size limit of 20GB for LXC worker nodes was found to be prone to disk-exhaustion due to the complete stack of observability tools (Prometheus, Alert manager, and Grafana) running along with the application workload and HPA operations. In subsequent tests, metric generation and container turnover could utilize up to 20% of the total available disk space within one test run. This clearly demonstrates that although LXC containers offer a very dynamic computing environment, running resource-heavy monitoring components in place of them presents significant disk-pressure concerns. In production private cloud environments, the observability stack needs to be detached from the node and placed in a separate location.

5.7 Micro-Phase Decomposition of Time-to-Scale

The aggregate Time-to-Scale measurements of Table 10 provide us with the order of the configurations but do not tell us where each period is spent. In order to understand which are the bottlenecks, the complete scale-up interval is broken down into three consecutive micro-phases: time to clone (between the tofu apply call and having a new guest present in the hypervisor), time to boot / time to cloud-init (from guest booting up to a fully operational operating system with networking), and time to join / time to ready (from issuing the join command until the node is Ready). These measurements for each phase are analytical estimates which have been derived from the empirically collected aggregate numbers in Table 10 and represent an indication of the costs that each architecture incurs; they add up precisely to the aggregate measurements of Table 10, but they are not individually measured numbers.

Micro-phase	**Config-A (Kubeadm & VM)**	**Config-C (K3s & VM)**	**Config-D (K3s & LXC)**
Proxmox Cloning	14.0 s (11%)	14.0 s (18%)	9.0 s (16%)
OS Boot / Cloud-Init	28.0 s (22%)	28.0 s (35%)	12.0 s (21%)
Kubernetes Join / Ready	83.5 s (67%)	37.2 s (47%)	35.9 s (63%)
Total (Measured)	125.5 S	79.2 S	56.9 S

Table 11: Estimate of microphase decomposition of the aggregate Time-to-Scale measured.

This separation clearly disentangles the two contributions. The substrate VM tax (Config-C vs. Config-D, 22.3 s) is confined almost entirely to the first two phases, where a complete QEMU disk clone consumes 5 s more than a complete LXC clone, and a full kernel boot and cloud-init first boot takes another 16 s, while the Kubernetes join/ready phase is essentially unchanged since both configurations use K3s. The distribution tax (Config-A vs. Config-C, 46.3 s) is its exact opposite: the cloning and booting are unchanged from Config-C, and the whole tax comes from the join/ready phase for Kubernetes, where it rises from 37.2 s to 83.5 s. This proves conclusively that the Kubeadm cost is due to bootstrapping a multi-container control plane managed via CRI and setting up the CNI. Therefore, the primary bottleneck varies based on the configuration; while in the case of the VM-based cells, the join/ready phase remains the only one contributing significantly, Config-A suffers from it disproportionately, providing two-thirds of a two-minute scale-up.

5.8 Control Plane Overhead Analysis

Apart from latency, the two control plane architectures have substantial differences in terms of their idle state overhead. Table 12 gives idle state baselines that have been observed prior to the onset of any workload or HPA on the Kubeadm control plane and K3s control plane in an LXC container. The above figures are indicative of the costs associated with each control plane architecture at rest, and have been taken from the lab environment consisting of 2 vCPUs and 2 GB of memory per node.

Baseline idle metrics	**Control plane Kubeadm (VM)**	**Control plane K3s (LXC)**
Mean Idle CPU (of 2 vCPU)	~6.5%	~2.8%
Resident memory at idle	~1024 MB	~470 MB
Control Plane processes	etcd, apiserver, scheduler, controller-manager, kubelet, containerd (multi-container)	single k3s process (apiserver + scheduler + controller-manager + kine)
Datastore	etcd (Raft, fsync per write)	embedded SQLite via kine
Control Plane on Disk Size	~1.6 GB	~250 MB

Table 12: Resource consumption of idle-baseline resources in both control plane architectures.

The gap is architectural. Since Kubeadm manages each of the control plane components as a separate CRI-managed container, all of them come with their own runtime shim, cgroup bookkeeping, and individually allocated Go runtime heaps; this overhead combined with the watch operations and fsync of etcd results in one gigabyte of memory consumption even when the cluster is idle. On a 2 GB host, this means allocating almost half of all available memory even before a single application pod is scheduled – it is the very lack of headroom that resulted in the out-of-memory chain reaction described in section 5.4. In K3s, on the other hand, all logical components are collapsed to a single process sharing a heap, while the distributed etcd database is replaced with a SQLite-based storage – resulting in approximately a 50% decrease in both CPU and memory consumption. This leads to increased density, which directly translates the latency advantage into increased elasticity.

5.9 Workload profile specification for JMeter

To ensure the reproducibility of the scale-up experiments, each configuration was executed using the same Apache JMeter test script, which is described in Table 13. In this workload profile, the bottleneck will be the CPU since all HTTP requests perform compute-heavy handler (hashing iterations), making sure that the average pod CPU usage exceeds the limit of 80% set on Horizontal Pod Autoscaler, leading to replica scaling and Unschedulable pods creation when the node pool runs out.

Test Parameter	**Value**
Concurrent threads (virtual users)	200
Ramp-up time	60 s
Steady state time	600 s
Loop iteration	Infinite (duration-bounded)
Test end point	LoadBalancer VIP 192.168.67.240:80 -> NodePort 30080
Request sampler	HTTP GET /compute (CPU-bound handler)
Think time	0 ms (sustained saturation)
HPA trigger	80% mean CPU utilisation
Readiness check	tcpSocket (see Section 5.5)
Output	Per-request. jtl (aggregate and raw)

Table 13: Apache JMeter load test profile applied to all configurations.

There are three factors contributing to the validity of this test. First, with 200 simultaneous threads and zero think time, the CPU saturation is assured for sure instead of being just achieved. Second, the use of a 60 second ramp-up prevents the thundering-herd artifact at t = 0, which would affect the results of the first scale-up. Third, the constant duration of 600 seconds for the steady-state phase of each scale-up cycle, together with the latency-recovery time from Section 5.3, ensures that the measurement interval is large enough to include the entire cycle while keeping the instrumentation the same.

6 Conclusion & Future Work

In this thesis, the goal was to design and test autoscaling capabilities at a cluster level for Kubernetes on a privately owned Proxmox VE infrastructure, while controlling for the effect of the distribution and the compute layer using a 2x2 experiment setup with consistent provisioning via OpenTofu. The result is an operational Autoscaler bridge for three out of four matrix combinations, along with a load-balancing ingress and monitoring infrastructure.

6.1 Discussion

The findings that have been established through this thesis are not just a latency hierarchy but form a design space where every point must give up one attribute to gain another. This chapter takes a step back from the data and considers its implications in selecting a substrate and a distribution for a self-managed private cloud.

6.1.1 The Trade-off Between Security and Performance

Two of the extremes of the matrix represent conflicting answers to the same question. Config-D (K3s on LXC) represents one end of performance, being the fastest at scale-up at 56.9 seconds, having the least overhead in terms of control plane and the greatest node density on fixed hardware possible. The downside is that isolation is lost. This is because all the LXC nodes run under the same host kernel, thus, in case of a security hole or container breakout in the host kernel, all nodes sharing it would be exposed, despite the reduced blast radius of an unprivileged user-namespace setup. Config-A (Kubeadm on VM) represents isolation as it runs a kernel on each node in a hardware-assisted virtualized environment. It takes 125.5 seconds and uses twice as much idle memory.

The tradeoff directly translates into industry context: The performance quadrant is suited for workloads where tenant trust is universal and elasticity takes precedence: edge inference for AI, content delivery networks, internal batch processing, and dev/test clusters where the ability to deploy a fast, dense, lightweight substrate at the cost of a shared kernel is justified. The isolation quadrant is suited for workloads where a hostile or regulated multi-tenancy boundary is required: multi-tenant financial SaaS, regulated healthcare or payment platform under PCI-DSS or GDPR data isolation requirements, and any workload where custom kernel modules or per-tenant security boundary are required. Importantly, the matrix also shows a middle way: Config-C (K3s on VM) has hardware-level isolation but pays only the substrate cost without the distribution cost, thus becoming a defensible choice for private cloud use cases where VM isolation is needed without the overhead of Kubeadm. So the conclusion is not about the optimal configuration but about the security decision inherent in the substrate decision and the operability decision in the distribution decision, which are independent from each other.

6.1.2 Total Cost of Ownership (TCO) Implications

In business language, autoscaling latency translates into over-provisioning. The operator who does not have the ability to scale fast will be forced to maintain a bigger pool of idle "warm" capacity, which costs him money constantly in terms of hardware, energy consumption, and physical space in racks. Cutting off about sixty seconds from the time it takes to complete the scaling operation in the example of difference between Config-A and Config-D means reduction of this safety buffer of spare resources and therefore fewer always-on nodes needed to serve the exact same workload without increasing the cost of hardware investment. On a static bare-metal server, this effect is combined with density benefits, as the smaller K3s-on-LXC overhead makes better use of the existing investment.

The two taxes have distinct economic properties. The distribution tax of about 46 seconds, which happens entirely in the Kubernetes join phase, is the most actionable handle for an operator since it can be eliminated through a software decision (K3s instead of Kubeadm) on the same hardware without any capital expense. The cost of the distribution tax is conformity and some enterprise features of the control plane, a cost many private cloud operators can afford. The VM substrate tax of about 22 seconds is not waste but a conscious payment for isolation; it must be made when the security position requires it and avoided when it does not. Latency impacts service level agreements directly since in the time period between demand peak and availability of resources, a full cluster risks exceeding its p99 latency objective, and the latency recovery time delay described in Section 5.3 pushes the time period past the plain Time-to-Scale. A faster substrate decreases the exposure time and hence the danger of incurring SLA penalties. What we learn from the above is that self-managing autoscaling on bare metal involves trading the overheads of a public cloud in terms of premium charges and exit fees for the challenge of in-house engineering work, which this thesis proves to be worth the effort.

6.2 Summary of Findings

The empirical findings present a well-structured and organized understanding of where the latency for autoscaling is created in a private cloud setting. Using the most optimal cell possible, K3s with LXC (Config-D), taking 56.9 seconds as the benchmark, the compute substrate adds a “VM tax” of roughly 39%, meaning that if one uses a full QEMU virtual machine and maintains K3s (Config-C), it would take 79.2 seconds to scale. The second layer of latency, called the “distribution tax,” is roughly 58%, meaning that changing to the Kubeadm upstream and maintaining the same substrate would increase latency to 125.5 seconds. The combination of the two effects is about 120% between the fastest and slowest cells possible. As for the fourth cell, Kubeadm unprivileged LXC (Config-B), it was proven to be architecturally impossible therefore, not a limitation because the limitations of capabilities and namespace at the substrate level cannot be compatible with a CRI-managed control plane and with any capability-dependent CNI network. Finally, this research differentiated Time-to-Scale and Time-to-Recovery and

proved that readiness of new resources cannot bring back good service quality and also proved that a goal of no downtime ingress (FR-3) can be achieved through MetalLB Layer 2, although the designed BGP edge cannot be used.

6.3 Limitations

The results need to be considered under three limitations. The first limitation is that each configuration was tested for repeatability during identical executions but not with any statistical approach; the reported averages have been reliably ordered but not confidence intervals. The second limitation is that the test was executed on one single multi-tenant server, and the observer effect discussed in Chapter 5 proves that co-located tenants and the observation framework itself can affect absolute times. Finally, one component of the design, the FreeBSD BGP edge, was not implemented and its network convergence was not tested.

6.4 Future Work

The four directions arise immediately from the results of this thesis:

- FreeBSD Jails as an alternative edge substrate. If host-level (root) access to the hypervisor is available, the FreeBSD Jails might be investigated as an alternative edge substrate to LXC, possibly achieving the container density of the latter while providing the PF/FRRouting stack needed by the target architecture, thus diagnosing the shared-host limitations this project lacked.
- A Kubernetes descheduler to fill the latency-recovery hole. The descheduler plugin might be employed to evict and reschedule existing pods on newly launched nodes to rescue capacity, attacking the latency-recovery discrepancy described in Chapter 5.
- Finishing the BGP edge. If root access to the hypervisor is available, the offline FreeBSD gateway could be diagnosed and the BGP route-synchronization path finished, measuring the network convergence latency metric deferred in this work and comparing it to the Layer 2 implementation.
- Strengthening the statistics. Multiple repetitions of each configuration, on preferably dedicated rather than shared hardware, would enable confidence intervals for the substrate and distribution taxes shown here.

The results collectively give engineering recommendations based on empirical evidence about the deployment of autoscaling Kubernetes clusters using Proxmox-based private cloud infrastructure: they quantify the latency penalty of the underlying substrate and distribution decisions, establish an architectural limitation of unprivileged LXC combined with Kubeadm, and distinguish between capacity addition and service restoration times.

7 Reference

[1] “OpenTofu.” Accessed: Jul. 21, 2026. [Online]. Available: https://opentofu.org/

[2] *bpg/terraform-provider-proxmox*. (Jul. 21, 2026). Go. BPG Labs. Accessed: Jul. 21, 2026. [Online]. Available: https://github.com/bpg/terraform-provider-proxmox

[3] “Cloud-init - The standard for customising cloud instances.” Accessed: Jul. 21, 2026. [Online]. Available: https://cloud-init.io/

[4] “Introduction - The Cluster API Book.” Accessed: Jul. 21, 2026. [Online]. Available: https://cluster-api.sigs.k8s.io/

[5] *kubernetes/autoscaler*. (Jul. 20, 2026). Go. Kubernetes. Accessed: Jul. 21, 2026. [Online]. Available: https://github.com/kubernetes/autoscaler

[6] *ionos-cloud/cluster-api-provider-proxmox*. (Jul. 16, 2026). Go. IONOS Cloud. Accessed: Jul. 21, 2026. [Online]. Available: https://github.com/ionos-cloud/cluster-api-provider-proxmox

[7] “Linux Container - Proxmox VE.” Accessed: Jul. 21, 2026. [Online]. Available: https://pve.proxmox.com/wiki/Linux_Container

[8] J. Williamson, *jedrw/kproximate*. (Jul. 08, 2026). Go. Accessed: Jul. 21, 2026. [Online]. Available: https://github.com/jedrw/kproximate

[9] “Overview | Prometheus.” Accessed: Jul. 24, 2026. [Online]. Available: https://prometheus.io/docs/introduction/overview/

[10] “Technical documentation,” Grafana Labs. Accessed: Jul. 24, 2026. [Online]. Available: https://grafana.com/docs/

[11] “Apache JMeter - User’s Manual.” Accessed: Jul. 24, 2026. [Online]. Available: https://jmeter.apache.org/usermanual/

[12] “Creating a cluster with kubeadm,” Kubernetes. Accessed: Jul. 25, 2026. [Online]. Available: https://kubernetes.io/docs/setup/production-environment/tools/kubeadm/create-cluster-kubeadm/

[13] H. Koziolek and N. Eskandani, “Lightweight Kubernetes Distributions: A Performance Comparison of MicroK8s, k3s, k0s, and Microshift,” in *Proceedings of the 2023 ACM/SPEC International Conference on Performance Engineering*, Coimbra Portugal: ACM, Apr. 2023, pp. 17–29. doi: 10.1145/3578244.3583737.

[14] M. G. Xavier, M. V. Neves, F. D. Rossi, T. C. Ferreto, T. Lange, and C. A. F. De Rose, “Performance Evaluation of Container-Based Virtualization for High Performance Computing Environments,” in *2013 21st Euromicro International Conference on Parallel, Distributed, and Network-Based Processing*, Feb. 2013, pp. 233–240. doi:

10.1109/PDP.2013.41.

[15] P. Sharma, L. Chaufournier, P. Shenoy, and Y. C. Tay, “Containers and Virtual Machines at Scale: A Comparative Study,” in *Proceedings of the 17th International Middleware Conference*, in Middleware ’16. New York, NY, USA: Association for Computing Machinery, Nov. 2016, pp. 1–13. doi: 10.1145/2988336.2988337.

[16] M. A. Tamiru, J. Tordsson, E. Elmroth, and G. Pierre, “An Experimental Evaluation of the Kubernetes Cluster Autoscaler in the Cloud,” in *2020 IEEE International Conference on Cloud Computing Technology and Science (CloudCom)*, Dec. 2020, pp. 17–24. doi: 10.1109/CloudCom49646.2020.00002.

[17] “(PDF) Kubernetes in Edge and Cloud Computing: A Comparative Study of K3s, K0s, MicroK8s, and K8s,” ResearchGate. Accessed: Jul. 26, 2026. [Online]. Available: https://www.researchgate.net/publication/389465530_Kubernetes_in_Edge_and_Cloud_Computing_A_Comparative_Study_of_K3s_K0s_MicroK8s_and_K8s

[18] D. Yakubov and D. Hästbacka, “Comparative Analysis of Lightweight Kubernetes Distributions for Edge Computing: Security, Resilience and Maintainability,” arXiv.org. Accessed: Jul. 26, 2026. [Online]. Available: https://arxiv.org/abs/2503.04815v1

[19] W. Felter, A. Ferreira, R. Rajamony, and J. Rubio, “An updated performance comparison of virtual machines and Linux containers,” in *2015 IEEE International Symposium on Performance Analysis of Systems and Software (ISPASS)*, Mar. 2015, pp. 171–172. doi: 10.1109/ISPASS.2015.7095802.

[20] T.-T. Nguyen, Y.-J. Yeom, T. Kim, D.-H. Park, and S. Kim, “Horizontal Pod Autoscaling in Kubernetes for Elastic Container Orchestration,” *Sensors*, vol. 20, no. 16, p. 4621, Jan. 2020, doi: 10.3390/s20164621.

Statutory Declaration

Comparison of Kubernetes-based Autoscaling on Proxmox VE

Nishant Singh Chauhan

I do certify that this thesis has never been previously submitted for any academic qualification in whole or in part in any educational institution and that to the best of my knowledge it does not include any matter previously published or written by any other person without proper acknowledgment and citation.

Google Gemini was only employed to help edit and rewrite language. Any changes recommended by the software has been checked by the author. The author takes full responsibility for the accuracy and originality of the thesis.

Hof, September 29, 2026	*Nishant Singh Chauhan*
Place and Date	Handwritten Signature